

Project-01

Building Scalable, Secure, and High-Performance
Web Applications with AWS 3-Tier Architecture

Submitted by: Navikaran

GitHub Repository: github.com/naviktg/navi

Region: Asia Pacific (Mumbai)

Contents

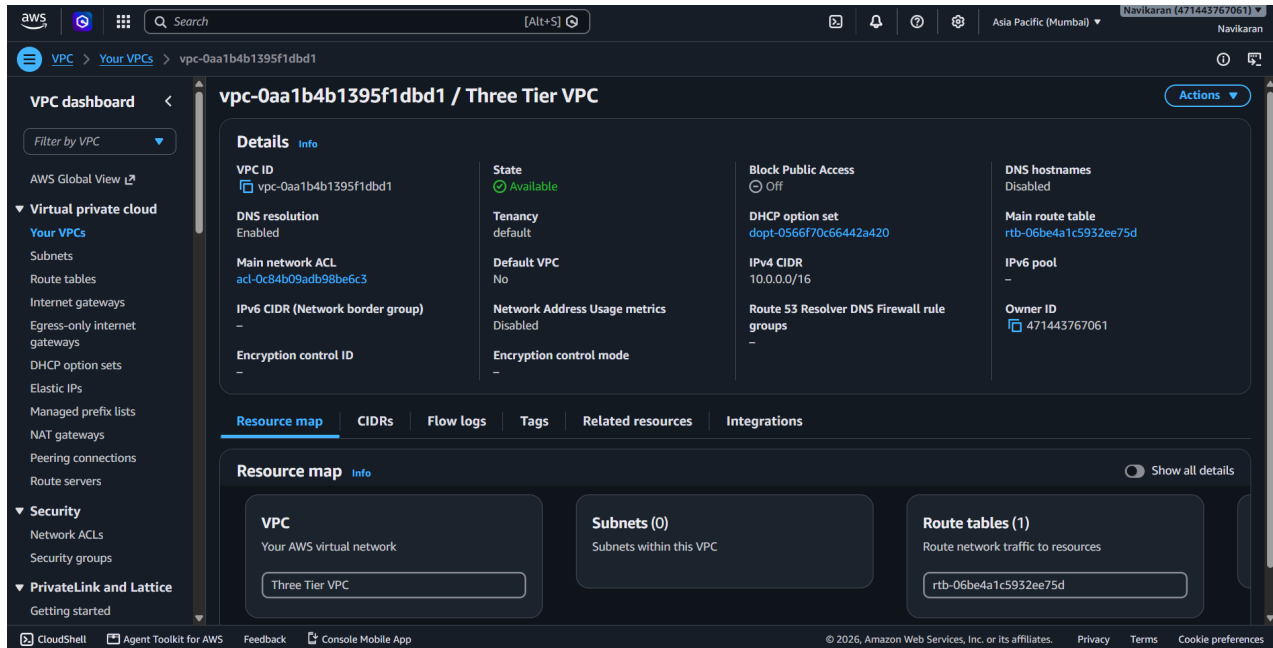
- Task 1 - VPC Networking (VPC, Subnets, IGW, NAT Gateway, Route Tables)
- Task 2 - Web Tier (Nginx on EC2)
- Task 3 - Application Tier (Flask on EC2)
- Task 4 - Database Tier (Amazon RDS MySQL)
- Task 5 - Security Groups (Web / App / DB / ALB tiers)
- Task 6 & 7 - Public & Internal Application Load Balancers
- Task 8 & 9 - Nginx Reverse Proxy + Flask-RDS Integration
- Task 10 - Full End-to-End Validation
- GitHub Submission & Final Live Testing

AWS 3-Tier Architecture Project

Task 1: Design and Configure AWS Networking Infrastructure

Step 1 - VPC Created

Custom VPC 'Three Tier VPC' created with CIDR block 10.0.0.0/16.



Step 2a - Public Subnet AZ1

Public subnet created: 10.0.1.0/24 (256 IPs).

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

Public AZ1

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.1.0/24 256 IPs

< > ^ v

Step 2b - Public Subnet AZ2

Public subnet created: 10.0.2.0/24 (256 IPs).

Subnet 2 of 2

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

Public AZ2

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.2.0/24256 IPs

Tags - optional

Key

Q Name

X

Value - optional

Q Public AZ2

X

Remove

Step 2c - App Subnet AZ1

Private App subnet created: 10.0.3.0/24 (256 IPs).

Subnet 3 of 3

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

App AZ1

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.3.0/24256 IPs

<>^v

▼ Tags - optional

Key

Value - optional

Q Name

Q App AZ1

Remove

Step 2d - App Subnet AZ2

Private App subnet created: 10.0.4.0/24 (256 IPs).

Subnet 4 of 4

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

App AZ2

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.4.0/24256 IPs

< > ^ v

▼ Tags - optional

Key

Value - optional

Q NameX

Q App AZ2X

Remove

Step 2e - DB Subnet AZ1

Private DB subnet created: 10.0.5.0/24 (256 IPs).

Subnet 5 of 5

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

DB AZ1

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.5.0/24256 IPs

<>^v

Tags - optional

Key

Value - optional

Q NameX

Q DB AZ1X

Remove

Step 2f - DB Subnet AZ2

Private DB subnet created: 10.0.6.0/24 (256 IPs).

Subnet 6 of 6

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

DB AZ2

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.6.0/24256 IPs

<>^v

Tags - optional

Key

Value - optional

Q NameX

Q DB AZ2X

Remove

Step 2 - All Subnets Overview

All 6 subnets (Public, App, DB across 2 AZs) listed and available.

VPC > Subnets

VPC dashboard

Filter by VPC

AWS Global View

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Subnets (6) Info

Last updated less than a minute ago

Actions

Create subnet

Find subnets by attribute or tag

Subnet ID : subnet-0456c6e2b7b1cbc48

Subnet ID : subnet-091f0137ccceffc90

Subnet ID : subnet-0cc46287ed7fe5616

Show more (+3)

Clear filters

	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	Public AZ2	subnet-091f0137ccceffc90	Available	vpc-0aa1b4b1395f1dbd1 Thre...	Off	10.0.2.0/24
☐	App AZ1	subnet-0cc46287ed7fe5616	Available	vpc-0aa1b4b1395f1dbd1 Thre...	Off	10.0.3.0/24
☐	DB AZ1	subnet-02befbc9155ac5f1e	Available	vpc-0aa1b4b1395f1dbd1 Thre...	Off	10.0.5.0/24
☐	App AZ2	subnet-0d093d42caf6ccd11	Available	vpc-0aa1b4b1395f1dbd1 Thre...	Off	10.0.4.0/24
☐	Public AZ1	subnet-0456c6e2b7b1cbc48	Available	vpc-0aa1b4b1395f1dbd1 Thre...	Off	10.0.1.0/24

Select a subnet

Step 3 - Creating Internet Gateway

Creating Internet Gateway named 'Internet Gateway Three Tier Project'.

Create internet gateway [Info](#)

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag

Creates a tag with a key of 'Name' and a value that you specify.

Internet Gateway Three Tier Project

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Value - optional

Q Name

×

Q Internet Gateway Three Tier Project

×

Remove

Add new tag

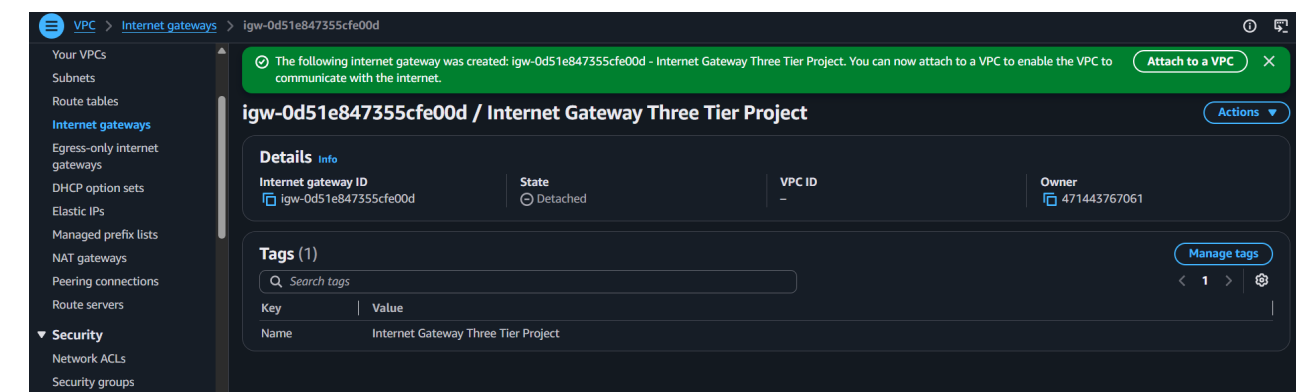
You can add 49 more tags.

Cancel

Create internet gateway

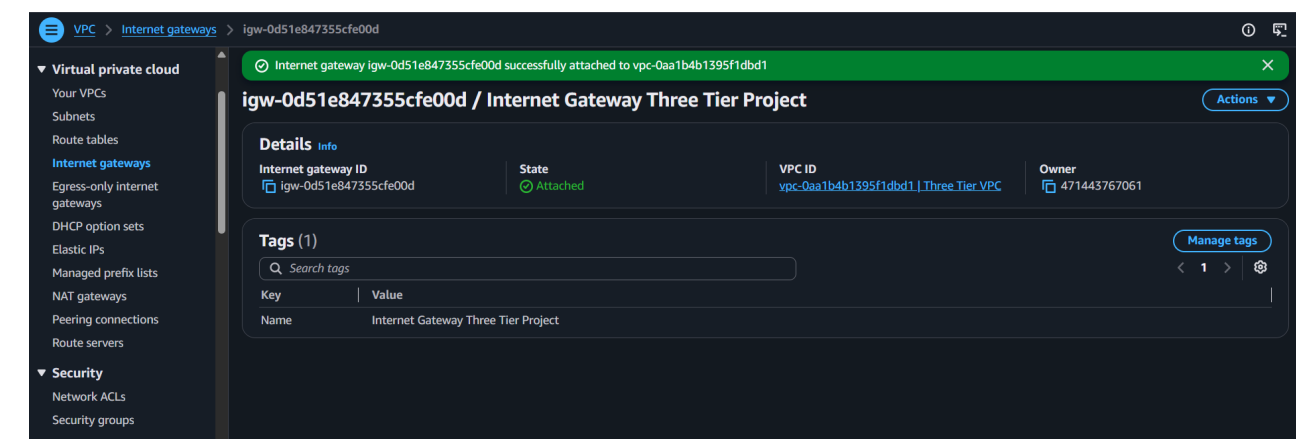
Step 3a - Internet Gateway Created

Internet Gateway created successfully, initially in Detached state.



Step 3b - IGW Attached to VPC

Internet Gateway successfully attached to the VPC.



Step 4a - Creating NAT Gateway

Allocating Elastic IP and creating NAT Gateway in Public Subnet AZ1.

Elastic IP address 13.205.228.8 (eipalloc-05e09349c88503541) allocated.

NAT gateway settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

NAT Gateway Three Tier Project

The name can be up to 256 characters long.

Availability mode

Info

Choose whether to deploy across all zones in the region or restrict to a single availability zone.

Regional - new

Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

Zonal

Provides granular control within a specific availability zone, adhering to subnet level settings.

Subnet

Select a subnet in which to create the NAT gateway.

subnet-0456c6e2b7b1cbc48 (Public AZ1)

Connectivity type

Select a connectivity type for the NAT gateway.

Public

Private

Elastic IP allocation ID

Info

Assign an Elastic IP address to the NAT gateway.

eipalloc-05e09349c88503541

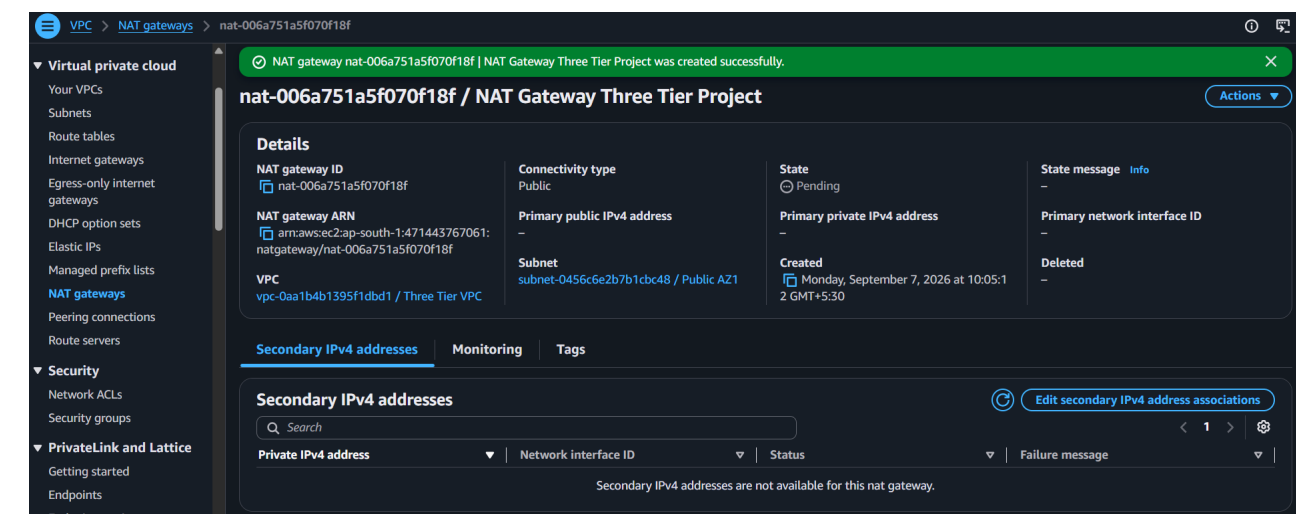
Allocate Elastic IP

Additional settings

Info

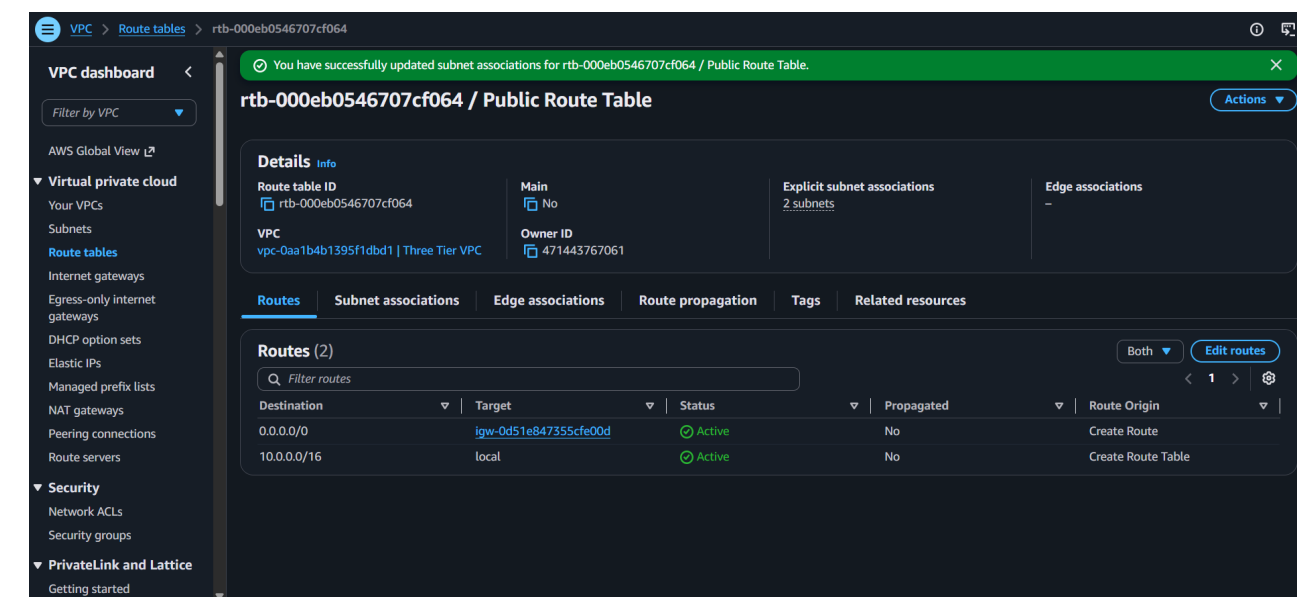
Step 4 - NAT Gateway Created

NAT Gateway 'NAT Gateway Three Tier Project' created in Public AZ1 subnet.



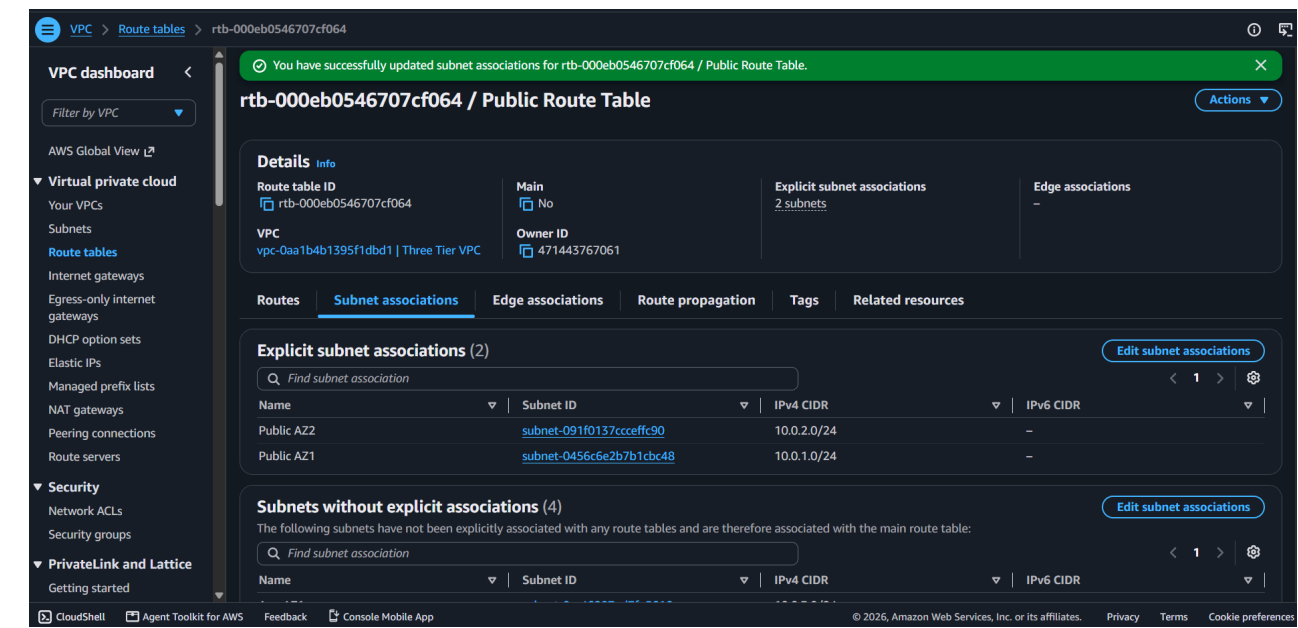
Step 5 - Public Route Table Routes

Public Route Table configured with route 0.0.0.0/0 -> Internet Gateway.



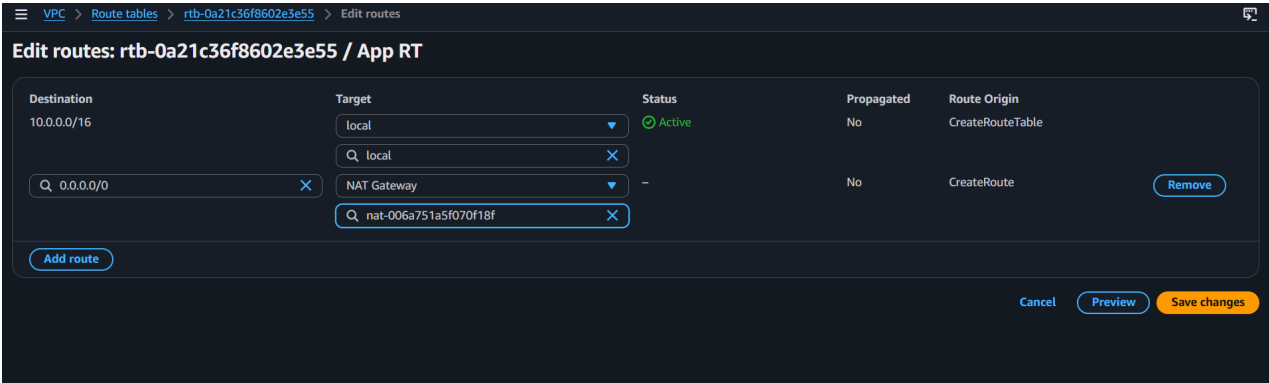
Step 5.1 - Public RT Subnet Associations

Public Route Table associated with Public AZ1 and Public AZ2 subnets.



Step 5.2a - App Route Table Routes

App Route Table configured with route 0.0.0.0/0 -> NAT Gateway.



Step 5.2b - App RT Subnet Associations

App Route Table associated with App AZ1 and App AZ2 subnets.

VPC dashboard

Filter by VPC

AWS Global View

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

You have successfully updated subnet associations for rtb-0a21c36f8602e3e55 / App RT.

rtb-0a21c36f8602e3e55 / App RT

Actions

Details

Route table ID

rtb-0a21c36f8602e3e55

VPC

vpc-0aa1b4b1395f1dbd1 | Three Tier VPC

Main

No

Owner ID

471443767061

Explicit subnet associations

2 subnets

Edge associations

-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Related resources

Explicit subnet associations (2)

Edit subnet associations

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
App AZ1	subnet-0cc46287ed7fe5616	10.0.3.0/24	-
App AZ2	subnet-0d093d42caf6ccd11	10.0.4.0/24	-

Subnets without explicit associations (2)

Edit subnet associations

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
------	-----------	-----------	-----------

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Step 5.3b - DB RT Subnet Associations

DB Route Table associated with DB AZ1 and DB AZ2 subnets (no internet route).

VPC

Route tables

rtb-03339d877a87c6c53

Filter by VPC

AWS Global View

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

You have successfully updated subnet associations for rtb-03339d877a87c6c53 / DB RT.

rtb-03339d877a87c6c53 / DB RT

Actions

Details

Info

Route table ID

rtb-03339d877a87c6c53

Main

No

Explicit subnet associations

2 subnets

Edge associations

-

VPC

vpc-0aa1b4b1395f1dbd1 | Three Tier VPC

Owner ID

471443767061

Routes

Subnet associations

Edge associations

Route propagation

Tags

Related resources

Explicit subnet associations (2)

Edit subnet associations

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
DB AZ1	subnet-02befbc9155ac5f1e	10.0.5.0/24	-
DB AZ2	subnet-0c7fd330aae4cf288	10.0.6.0/24	-

Subnets without explicit associations (0)

Edit subnet associations

Find subnet association

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
------	-----------	-----------	-----------

AWS 3-Tier Architecture Project

Task 2: Deploy the Web Tier (Presentation Layer)

Step 1 - Web Tier EC2 Instance Launched

EC2 instance 'Public AZ1' launched and Running, in Public AZ1 subnet with public IP 13.203.72.19.

The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows the navigation menu with categories like EC2, Images, and Elastic Block Store. The main content area is titled 'Instance summary for i-07d2fab01e21fe5ba (Public AZ1)'. It provides a comprehensive overview of the instance's configuration, including its ID, IP addresses, state, DNS names, and various identifiers like VPC ID and Subnet ID. The instance is currently in a 'Running' state.

Property	Value
Instance ID	i-07d2fab01e21fe5ba
Public IPv4 address	13.203.72.19 open address
Private IPv4 addresses	10.0.1.125
Instance state	Running
Public DNS	ec2-13-203-72-19.ap-south-1.compute.amazonaws.com open address
Private IP DNS name (IPv4 only)	ip-10-0-1-125.ap-south-1.compute.internal
Instance type	t3.micro
VPC ID	vpc-0aa1b4b1395f1dbd1 (Three Tier VPC) link
Subnet ID	subnet-0456c6e2b7b1cbc48 (Public AZ1) link
Instance ARN	arn:aws:ec2:ap-south-1:471443767061:instance/i-07d2fab01e21fe5ba
Auto-assigned IP address	13.203.72.19 [Public IP]
Hostnames	IP name: ip-10-0-1-125.ap-south-1.compute.internal
Answer private resource DNS name	-
IAM role	-
IMDSv2	Required
Operator	-
AWS Compute Optimizer finding	Opt-in to AWS Compute Optimizer for recommendation s. Learn more
Elastic IP addresses	-
Auto Scaling Group name	-
Managed	false

Step 2 - Connected & Installed Nginx

Connected via SSH, ran yum update, installed Nginx, enabled and started the service. systemctl status confirms nginx is active (running). Local curl test returns HTTP/1.1 200 OK.

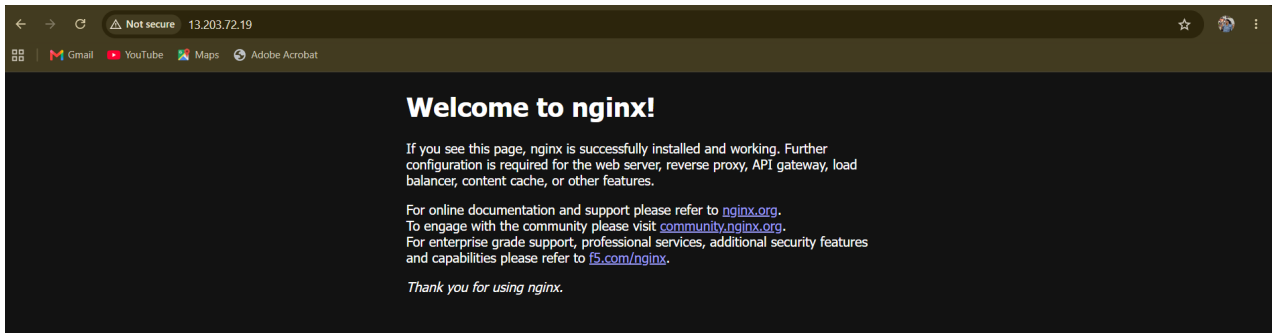
```
[ec2-user@ip-10-0-1-125 ~]$ nano Putty.pem
[ec2-user@ip-10-0-1-125 ~]$ chmod 400 Putty.pem
[ec2-user@ip-10-0-1-125 ~]$ ssh -i Putty.pem ec2-user@13.203.72.19

^C
[ec2-user@ip-10-0-1-125 ~]$ ls
Putty.pem
[ec2-user@ip-10-0-1-125 ~]$ sudo yum update -y
Last metadata expiration check: 0:20:47 ago on Mon Sep  7 05:35:51 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-125 ~]$ sudo yum install nginx -y
Last metadata expiration check: 0:21:13 ago on Mon Sep  7 05:35:51 2026.
Package nginx-1:1.30.4-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-125 ~]$ sudo systemctl enable nginx
[ec2-user@ip-10-0-1-125 ~]$ sudo systemctl start nginx
[ec2-user@ip-10-0-1-125 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-09-07 05:37:15 UTC; 21min ago
     Main PID: 26413 (nginx)
       Tasks: 3 (limit: 1059)
        Memory: 3.5M
         CPU: 67ms
    CGroup: /system.slice/nginx.service
            └─26413 "nginx: master process /usr/sbin/nginx"
              └─26414 "nginx: worker process"
                └─26415 "nginx: worker process"

Sep 07 05:37:15 ip-10-0-1-125.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 07 05:37:15 ip-10-0-1-125.ap-south-1.compute.internal nginx[26411]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 07 05:37:15 ip-10-0-1-125.ap-south-1.compute.internal nginx[26411]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 07 05:37:15 ip-10-0-1-125.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-10-0-1-125 ~]$ curl -I http://local host
curl: (6) Could not resolve hosts: local
curl: (6) Could not resolve host: host
[ec2-user@ip-10-0-1-125 ~]$ cat /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1        localhost6 localhost6.localdomain6
[ec2-user@ip-10-0-1-125 ~]$ curl -I http://127.0.0.1
HTTP/1.1 200 OK
Server: nginx/1.30.4
Date: Mon, 07 Sep 2026 06:10:54 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Mon, 20 Jul 2026 22:30:14 GMT
Connection: keep-alive
ETag: "6a5eal76-380"
Accept-Ranges: bytes
```

Step 3 - Verified Web Page Access

Opened the instance's public IP (13.203.72.19) in browser - default Nginx welcome page loads successfully, confirming the Web Tier is live.



AWS 3-Tier Architecture Project

Task 3: Deploy the Application Tier (Backend API)

Step 1 - App Tier EC2 Instance Launched

EC2 instance 'App AZ1' launched and Running in the private App subnet (10.0.3.225), no public IP - accessed via Session Manager (SSM).

Instance summary for i-070d82f7ce0d3c034 (App AZ1) Info

Updated 6 minutes ago

Instance ID

 i-070d82f7ce0d3c034

IPv6 address

–

Hostname type

IP name: ip-10-0-3-225.ap-south-1.compute.internal

Answer private resource DNS name

–

Auto-assigned IP address

–

IAM role

–

IMDSv2

Required

Operator

–

Public IPv4 address

–

Instance state

 Running

Private IP DNS name (IPv4 only)

 ip-10-0-3-225.ap-south-1.compute.internal

Instance type

t3.micro

VPC ID

 vpc-0aa1b4b1395f1dbd1 (Three Tier VPC) [↗](#)

Subnet ID

 subnet-0cc46287ed7fe5616 (App AZ1) [↗](#)

Instance ARN

 arn:aws:ec2:ap-south-1:471443767061:instance/i-070d82f7ce0d3c034

Private IPv4 addresses

 10.0.3.225

Public DNS

–

Elastic IP addresses

–

AWS Compute Optimizer finding

 Opt-in to AWS Compute Optimizer for recommendation s. [Learn more ↗](#)

Auto Scaling Group name

–

Managed

false

Connect [↗](#)

Instance state [▼](#)

Actions [▼](#)

Step 2 - Installed Python3 & pip

Connected via SSM and ran 'sudo yum install python3 python3-pip -y' - Python3 and pip3 installed successfully.

```
sh-5.2$ sudo yum install python3 python3-pip -y
Amazon Linux 2023 Kernel Livepatch repository                               500 kB/s | 69 kB    00:00
Last metadata expiration check: 0:00:01 ago on Mon Sep  7 06:53:33 2026.
Package python3-3.9.25-1.amzn2023.0.9.x86_64 is already installed.
Dependencies resolved.

=====
Package                                Architecture      Version              Repository            Size
=====
Installing:
python3-pip                             noarch             21.3.1-2.amzn2023.0.21  amazonlinux            1.8 M
Installing weak dependencies:
libxcrypt-compat                       x86_64             4.4.33-7.amzn2023      amazonlinux             92 k
=====
Transaction Summary
=====
Install 2 Packages

Total download size: 1.9 M
Installed size: 11 M
Downloading Packages:
(1/2): libxcrypt-compat-4.4.33-7.amzn2023.x86_64.rpm                    2.0 MB/s | 92 kB    00:00
(2/2): python3-pip-21.3.1-2.amzn2023.0.21.noarch.rpm                  29 MB/s | 1.8 MB    00:00
-----
Total                                                                    18 MB/s | 1.9 MB    00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :                                                                1/1
Installing : libxcrypt-compat-4.4.33-7.amzn2023.x86_64                  1/2
Installing : python3-pip-21.3.1-2.amzn2023.0.21.noarch                  2/2
Running scriptlet: python3-pip-21.3.1-2.amzn2023.0.21.noarch            2/2
Verifying  : libxcrypt-compat-4.4.33-7.amzn2023.x86_64                  1/2
Verifying  : python3-pip-21.3.1-2.amzn2023.0.21.noarch                  2/2
Installed:
libxcrypt-compat-4.4.33-7.amzn2023.x86_64                               python3-pip-21.3.1-2.amzn2023.0.21.noarch
```

Step 3 - Installed Git

Installed git and its dependencies (git-core, perl-Git, etc.) to clone the application repository.

Package	Architecture	Version	Repository	Size
Installing:				
git	x86_64	2.50.1-1.amzn2023.0.1	amazonlinux	53 k
Installing dependencies:				
git-core	x86_64	2.50.1-1.amzn2023.0.1	amazonlinux	4.9 M
git-core-doc	noarch	2.50.1-1.amzn2023.0.1	amazonlinux	2.8 M
perl-Error	noarch	1:0.17030-2.amzn2023.0.1	amazonlinux	42 k
perl-File-Find	noarch	1.37-477.amzn2023.0.9	amazonlinux	26 k
perl-Git	noarch	2.50.1-1.amzn2023.0.1	amazonlinux	41 k
perl-TermReadKey	x86_64	2.38-9.amzn2023.0.3	amazonlinux	36 k
perl-lib	x86_64	0.65-477.amzn2023.0.9	amazonlinux	15 k
Transaction Summary				
Install 8 Packages				
Total download size: 7.9 M				
Installed size: 41 M				
Downloading Packages:				
(1/8): git-2.50.1-1.amzn2023.0.1.x86_64.rpm			1.3 MB/s 53 kB	00:00
(2/8): git-core-doc-2.50.1-1.amzn2023.0.1.noarch.rpm			37 MB/s 2.8 MB	00:00
(3/8): perl-Error-0.17030-2.amzn2023.0.1.noarch.rpm			1.0 MB/s 42 kB	00:00
(4/8): git-core-2.50.1-1.amzn2023.0.1.x86_64.rpm			47 MB/s 4.9 MB	00:00
(5/8): perl-File-Find-1.37-477.amzn2023.0.9.noarch.rpm			880 kB/s 26 kB	00:00
(6/8): perl-Git-2.50.1-1.amzn2023.0.1.noarch.rpm			1.4 MB/s 41 kB	00:00
(7/8): perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64.rpm			1.4 MB/s 36 kB	00:00
(8/8): perl-lib-0.65-477.amzn2023.0.9.x86_64.rpm			633 kB/s 15 kB	00:00
Total			45 MB/s 7.9 MB	00:00
Running transaction check				
Transaction check succeeded.				
Running transaction test				
Transaction test succeeded.				
Running transaction				
Preparing	:			1/1
Installing	:	git-core-2.50.1-1.amzn2023.0.1.x86_64		1/8
Installing	:	git-core-doc-2.50.1-1.amzn2023.0.1.noarch		2/8

Step 4 - Installed Flask & MySQL Connector

Ran 'pip3 install flask flask-mysql mysql-connector-python' - all packages downloaded and installed successfully for the ec2-user.

```
sh-5.2$ pip3 install flask flask-mysql mysql-connector-python
Defaulting to user installation because normal site-packages is not writeable
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
    |#####| 103 kB 19.5 MB/s
Collecting flask-mysql
  Downloading Flask_MySQL-1.6.0-py2.py3-none-any.whl (3.8 kB)
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.4.0-cp39-cp39-manylinux_2_28_x86_64.whl (33.9 MB)
    |#####| 33.9 MB 51.1 MB/s
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.3-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.0-py3-none-any.whl (226 kB)
    |#####| 226 kB 74.8 MB/s
Collecting importlib-metadata>=3.6.0
  Downloading importlib_metadata-8.7.1-py3-none-any.whl (27 kB)
Collecting click>=8.1.3
  Downloading click-8.1.8-py3-none-any.whl (98 kB)
    |#####| 98 kB 20.1 MB/s
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
    |#####| 134 kB 92.0 MB/s
Collecting PyMySQL
  Downloading pymysql-1.2.0-py3-none-any.whl (45 kB)
    |#####| 45 kB 10.4 MB/s
Collecting zipp>=3.20
  Downloading zipp-3.23.1-py3-none-any.whl (10 kB)
Installing collected packages: zipp, markupsafe, werkzeug, jinja2, itsdangerous, importlib-metadata, click, blinker, PyMySQL, flask, mysql-connector-python, flask-mysql
WARNING: The script flask is installed in '/home/ssm-user/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed PyMySQL-1.2.0 blinker-1.9.0 click-8.1.8 flask-3.1.3 flask-mysql-1.6.0 importlib-metadata-8.7.1 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 mysql-connector-python-9.4.0 werkzeug-3.1.8 zipp-3.23.1
sh-5.2$
```

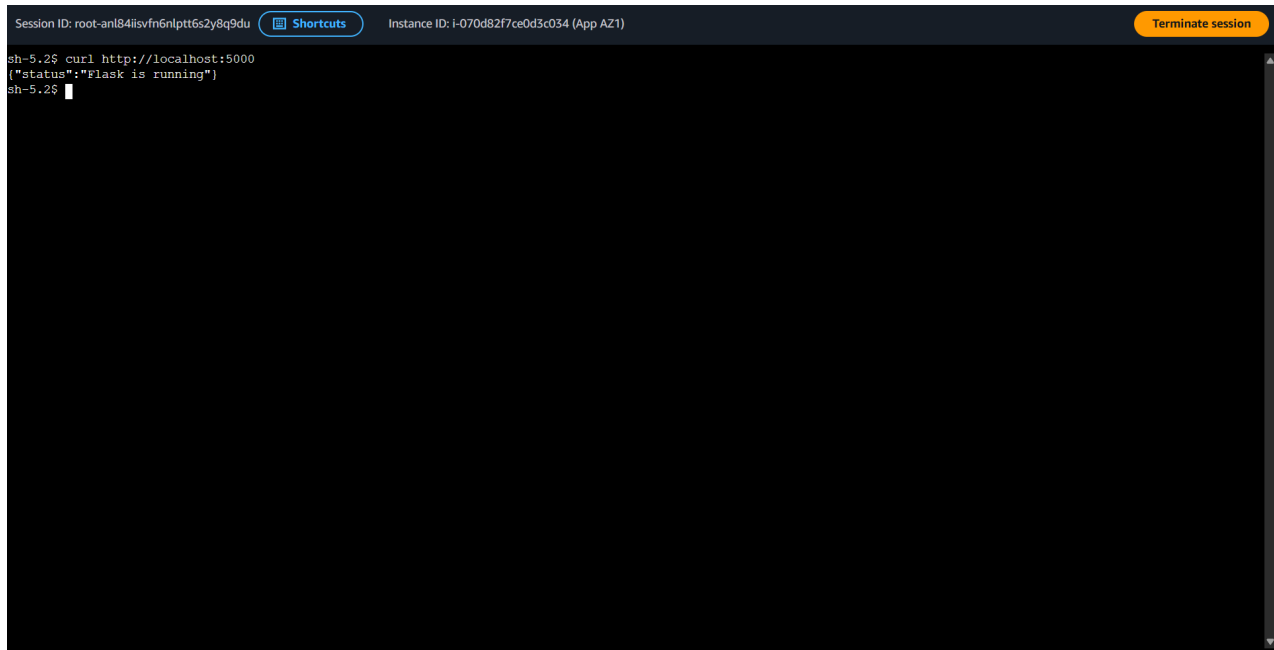
Step 5 - Cloned Repo & Ran Flask App

Cloned the Three-Tier-Architecture repo (app.py, App.md, DB.md, nginx.conf, proxy.md), then ran 'python3 app.py' - Flask started serving on http://127.0.0.1:5000 (all addresses 0.0.0.0).

```
drwxr-xr-x. 7 ssm-user ssm-user 147 Sep  7 07:11 .git
-rw-r--r--. 1 ssm-user ssm-user 1823 Sep  7 07:11 App.md
-rw-r--r--. 1 ssm-user ssm-user 2597 Sep  7 07:11 DB.md
-rw-r--r--. 1 ssm-user ssm-user 869 Sep  7 07:11 app.py
-rw-r--r--. 1 ssm-user ssm-user 1349 Sep  7 07:11 index.html
-rw-r--r--. 1 ssm-user ssm-user 619 Sep  7 07:11 nginx.conf
-rw-r--r--. 1 ssm-user ssm-user 2158 Sep  7 07:11 proxy.md
sh-5.2$ find ~ -maxdepth 3 -type d -name "Three-Tier-Architecture"
>
> cd ~
> ^C
sh-5.2$ cd ~
sh-5.2$ git clone https://github.com/Vennilavanguvi/Three-Tier-Architecture.git
fatal: destination path 'Three-Tier-Architecture' already exists and is not an empty directory.
sh-5.2$ cd Three-Tier-Architecture
sh-5.2$ ls
App.md DB.md app.py index.html nginx.conf proxy.md
sh-5.2$ pip3 install flask flask-mysql mysql-connector-python
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: flask in /home/ssm-user/.local/lib/python3.9/site-packages (3.1.3)
Requirement already satisfied: flask-mysql in /home/ssm-user/.local/lib/python3.9/site-packages (1.6.0)
Requirement already satisfied: mysql-connector-python in /home/ssm-user/.local/lib/python3.9/site-packages (9.4.0)
Requirement already satisfied: Jinja2>=3.1.2 in /home/ssm-user/.local/lib/python3.9/site-packages (from flask) (3.1.6)
Requirement already satisfied: click>=8.1.3 in /home/ssm-user/.local/lib/python3.9/site-packages (from flask) (8.1.8)
Requirement already satisfied: blinker>=1.9.0 in /home/ssm-user/.local/lib/python3.9/site-packages (from flask) (1.9.0)
Requirement already satisfied: itsdangerous>=2.2.0 in /home/ssm-user/.local/lib/python3.9/site-packages (from flask) (2.2.0)
Requirement already satisfied: Werkzeug>=3.1.0 in /home/ssm-user/.local/lib/python3.9/site-packages (from flask) (3.1.8)
Requirement already satisfied: MarkupSafe>=2.1.1 in /home/ssm-user/.local/lib/python3.9/site-packages (from flask) (3.0.3)
Requirement already satisfied: importlib-metadata>=3.6.0 in /home/ssm-user/.local/lib/python3.9/site-packages (from flask) (8.7.1)
Requirement already satisfied: PyMySQL in /home/ssm-user/.local/lib/python3.9/site-packages (from flask-mysql) (1.2.0)
Requirement already satisfied: zipp>=3.2.0 in /home/ssm-user/.local/lib/python3.9/site-packages (from importlib-metadata>=3.6.0->flask) (3.23.1)
sh-5.2$ python app.py
sh: python: command not found
sh-5.2$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
```

Step 6 - Verified Flask App Running

Ran 'curl http://localhost:5000' - received {"status": "Flask is running"}, confirming the Application Tier is live.

A screenshot of a terminal window with a dark background. The top bar shows session and instance IDs, a 'Shortcuts' button, and a 'Terminate session' button. The terminal text shows a shell prompt 'sh-5.2\$' followed by the command 'curl http://localhost:5000' and its output '{"status": "Flask is running"}'.

```
Session ID: root-anl84iisvfn6nptt6s2y8q9du Shortcuts Instance ID: i-070d82f7ce0d3c034 (App AZ1) Terminate session
sh-5.2$ curl http://localhost:5000
{"status": "Flask is running"}
sh-5.2$
```


AWS 3-Tier Architecture Project

Task 4: Deploy and Configure the Database Tier (RDS MySQL)

Step 1 - Create RDS Database

RDS Console -> Create database. Engine type: MySQL, method: Full configuration, template: Dev/Test.

Engine type Info

☐ Aurora (MySQL Compatible) ☐ Aurora (PostgreSQL Compatible) ☒ MySQL ☐ PostgreSQL

☐ MariaDB ☐ Oracle ☐ Microsoft SQL Server ☐ IBM Db2

Choose a database creation method

☒ Full configuration
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ Easy create
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Templates
Choose a sample template to meet your use case.

☐ Production
Use defaults for high availability and fast, consistent performance.

☒ Dev/Test
This instance is intended for development use outside of a production environment.

☐ Sandbox
To develop new applications, test existing applications, or gain hands-on experience with Amazon RDS.

Connectivity

DB subnet group

Choose the DB subnet group. A DB subnet group defines which subnets in a virtual private cloud (VPC) are designated for your database.

A DB subnet group has subnets in at least two Availability Zones in its AWS Region. The subnets in a DB subnet group are either public or private, depending on the configuration of their network ACLs and routing tables.

For security, the subnets in a DB subnet group are typically private. If you set up connectivity with a compute resource during database creation, RDS creates a DB subnet group with private subnets, and the setting can't be changed.

If the database must be publicly accessible, all of the subnets in its DB subnet group must be public.

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Step 2 - Select VPC & DB Subnet Group

Selected 'Three Tier VPC' (6 subnets, 3 AZs) and chose to create a new DB Subnet Group for the database's connectivity settings.

Virtual private cloud (VPC)

Info

Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Three Tier VPC (vpc-0aa1b4b1395f1dbd1)

6 Subnets, 3 Availability Zones

After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

ⓘ After a database is created, you can't change its VPC.

DB subnet group

Info

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

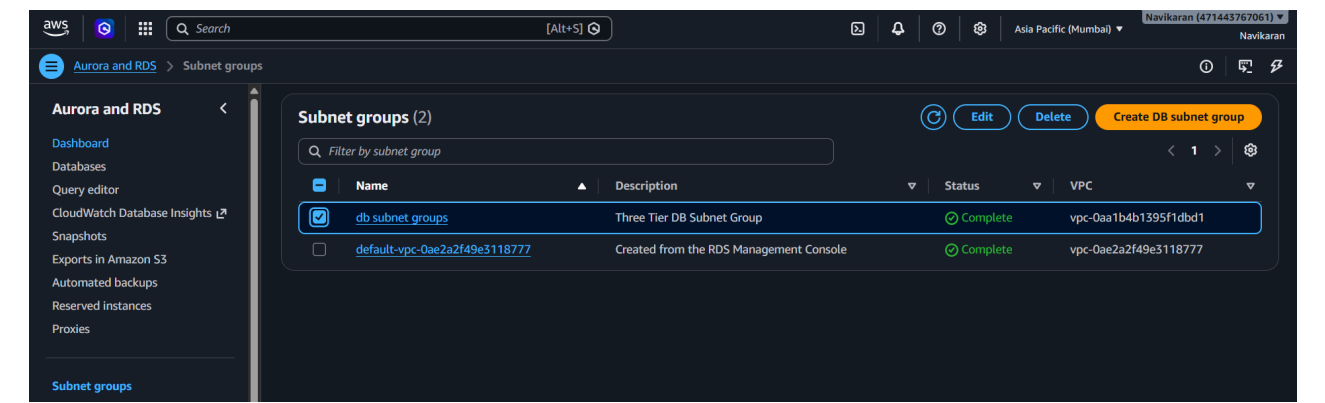
Create new DB Subnet Group

Create new DB Subnet Group

Yes

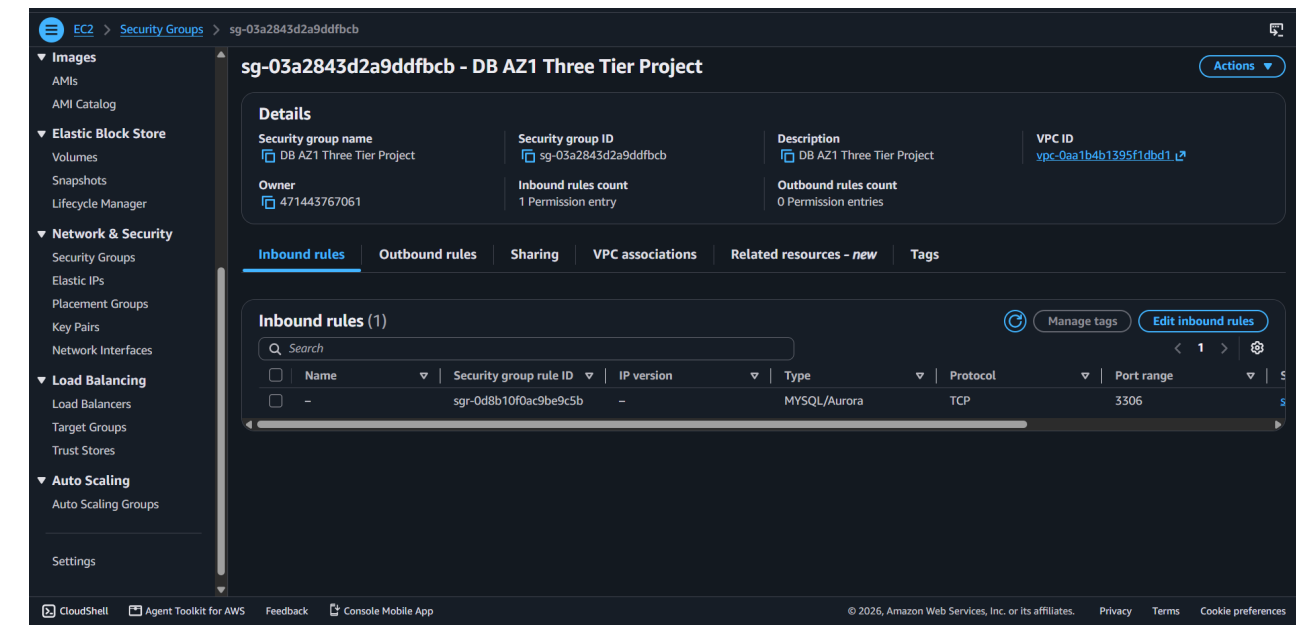
Step 3 - DB Subnet Group Created

'db subnet groups' (Three Tier DB Subnet Group) created and shows Complete status, associated with the Three Tier VPC.



Step 4 - DB Security Group

Created 'DB AZ1 Three Tier Project' security group with one inbound rule allowing MySQL/Aurora (TCP 3306) - restricted to the App Tier only.



Step 5 - Connect to RDS & Create Database/Table

From the App EC2 terminal, connected to the RDS endpoint via mysql client, ran CREATE DATABASE feedbackdb; then CREATE TABLE feedback (id, name, email, feedback, created).

```
---

## Architecture Flow

```text
Flask Application (Private EC2)
 |
 | INSERT Feedback
 v
Amazon RDS MySQL (Private Subnet)
 |
 | feedbackdb.feedback
 v
...
sh-5.2$ mysql -h db.ctq00g6wq6xj.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MySQL connection id is 49
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
MySQL [(none)]> CREATE DATABASE feedbackdb;
Query OK, 1 row affected (0.005 sec)

MySQL [(none)]> USE feedbackdb;
Database changed
MySQL [feedbackdb]> CREATE TABLE feedback (
 -> id INT AUTO-INCREMENT PRIMARY KEY,
 -> name VARCHAR(100),
 -> email VARCHAR(100),
 -> email VARCHAR(100),
 -> feedback TEXT,
 -> created TIMESTAMP DEFAULT CURRENT_TIMESTAMP
 ->);
```

## Step 6 - Verified Table Structure

Fixed a syntax error in the CREATE TABLE statement, then confirmed with SHOW TABLES and DESCRIBE feedback - table structure (id, name, email, feedback, created) confirmed correct.

```
-> MySQL [feedbackdb]>
->);
->);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'MySQL [feedbackdb
]> once more
-> CREATE TABLE feedback (
MySQL [feedbackdb]>
);
^ at line 1
MySQL [feedbackdb]> CREATE TABLE feedback (
-> id INT AUTO INCREMENT PRIMARY KEY,
-> name VARCHAR(100),
-> email VARCHAR(100),
-> feedback TEXT,
-> created TIMESTAMP DEFAULT CURRENT_TIMESTAMP
->);
Query OK, 0 rows affected (0.022 sec)

MySQL [feedbackdb]> SHOW TABLES;
+-----+
| Tables_in_feedbackdb |
+-----+
| feedback |
+-----+
1 row in set (0.001 sec)

MySQL [feedbackdb]> DESCRIBE feedback;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int | NO | PRI | NULL | auto_increment |
| name | varchar(100) | YES | | NULL | |
| email | varchar(100) | YES | | NULL | |
| feedback | text | YES | | NULL | |
| created | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)

MySQL [feedbackdb]>
```

## Step 7 - Connected Flask App to RDS

Updated app.py with RDS connection details, resolved a 'pymysql not defined' import error, then restarted the app - Flask now running on http://10.0.3.225:5000 (App Tier private IP).

```
| name | varchar(100) | YES | | NULL | |
| email | varchar(100) | YES | | NULL | |
| feedback | text | YES | | NULL | |
| created | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)

MySQL [feedbackdb]> exit
Bye
sh-5.2$ ~/app.py
sh: /home/ssm-user/app.py: No such file or directory
sh-5.2$ nano ~/app.py
sh-5.2$ pkill -f "python3 app.py"
sh-5.2$ python3 ~/app.py &
[1] 33580
sh-5.2$ Traceback (most recent call last):
 File "/home/ssm-user/app.py", line 1, in <module>
 db = pymysql.connect(
NameError: name 'pymysql' is not defined
python3 ~/app.py &
[2] 33646
[1] Done(1) python3 ~/app.py
sh-5.2$ Traceback (most recent call last):
 File "/home/ssm-user/app.py", line 1, in <module>
 db = pymysql.connect(
NameError: name 'pymysql' is not defined
python3 nano ~/app.py
[2]+ Done(1) python3 ~/app.py
sh-5.2$ pkill -f "python3 app.py"
sh-5.2$ python3 ~/app.py &
[1] 33710
sh-5.2$ * Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.3.225:5000
Press CTRL+C to quit
```

## Step 8 - End-to-End Test Successful

Submitted test feedback via curl POST to /submit -> {"message": "Feedback submitted successfully!"}.  
Connected to RDS and ran SELECT \* FROM feedback - the submitted row (Navi, navi@example.com, 'Testing from App EC2') is confirmed stored in the database.

```
20- (data['name'], data['email'], data['feedback'])
21-)
22- db.commit()
23- return jsonify({"message": "Feedback submitted successfully!"})
24-
25-if __name__ == '__main__':
26- app.run(host='0.0.0.0', port=5000)
(2)~ Done(1) python3 ~/app.py
sh-5.2$ curl -X POST ...
curl: (6) Could not resolve host: ...
sh-5.2$ curl -X POST http://localhost:5000/submit \
> -H "Content-Type: application/json" \
> -d '{"name":"Navi","email":"navi@example.com","feedback":"Testing from App EC2"}'
127.0.0.1 - - [07/Sep/2026 10:05:35] "POST /submit HTTP/1.1" 200 -
{"message": "Feedback submitted successfully!"}
sh-5.2$ mysql -h db.ctq00g6wq6xj.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MySQL connection id is 77
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> USE feedbackdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [feedbackdb]> SELECT * FROM feedback;
+----+-----+-----+-----+-----+
| id | name | email | feedback | created |
+----+-----+-----+-----+-----+
| 1 | Navi | navi@example.com | Testing from App EC2 | 2026-09-07 10:05:35 |
+----+-----+-----+-----+-----+
1 row in set (0.001 sec)

MySQL [feedbackdb]>
```

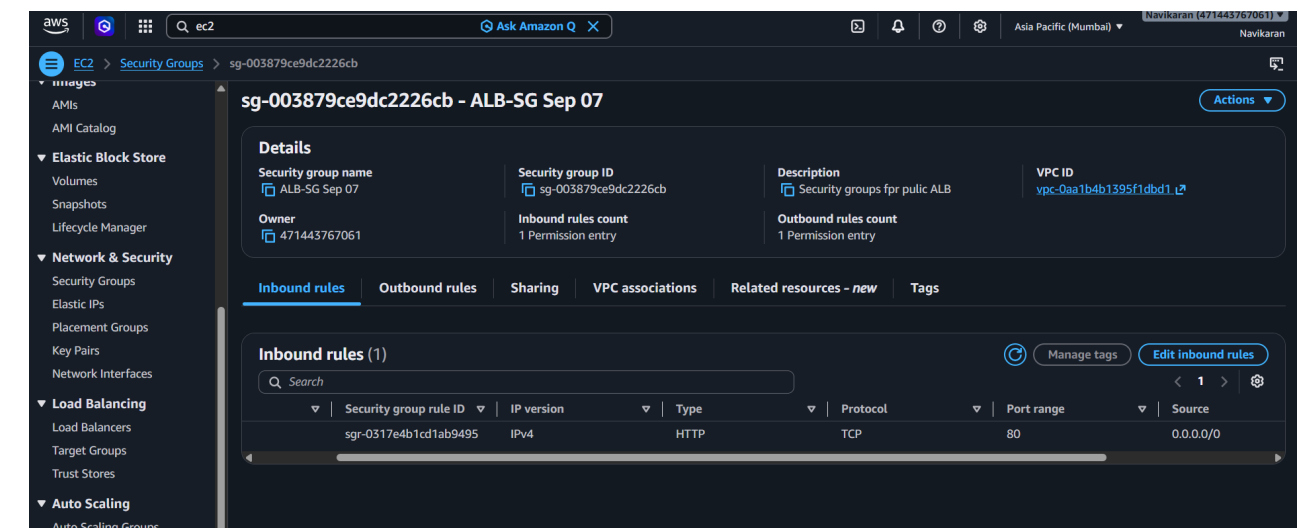


# AWS 3-Tier Architecture Project

## Task 5: Implement Network and Instance Security

### Step 1 - ALB Security Group

'ALB-SG Sep 07' created for the public ALB - inbound rule allows HTTP (TCP 80) from 0.0.0.0/0 (internet).



# Step 2 - Web Tier Security Group

'Public AZ1 Three Tier Project' SG has 2 inbound rules: HTTP (80) from the ALB-SG only, and SSH (22) restricted to the admin's IP.

aws

ec2

Ask Amazon Q

Asia Pacific (Mumbai)

Navikaran (471443767061)

EC2 > Security Groups > sg-0721baaa8a3eddda5

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

sg-0721baaa8a3eddda5 - Public AZ1 Three Tier Project

Actions

Details

Security group name

Public AZ1 Three Tier Project

Security group ID

sg-0721baaa8a3eddda5

Description

launch-wizard-1 created 2026-09-07T04:51:37.515Z

VPC ID

vpc-0aa1b4b1395f1dbd1

Owner

471443767061

Inbound rules count

2 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Manage tags

Edit inbound rules

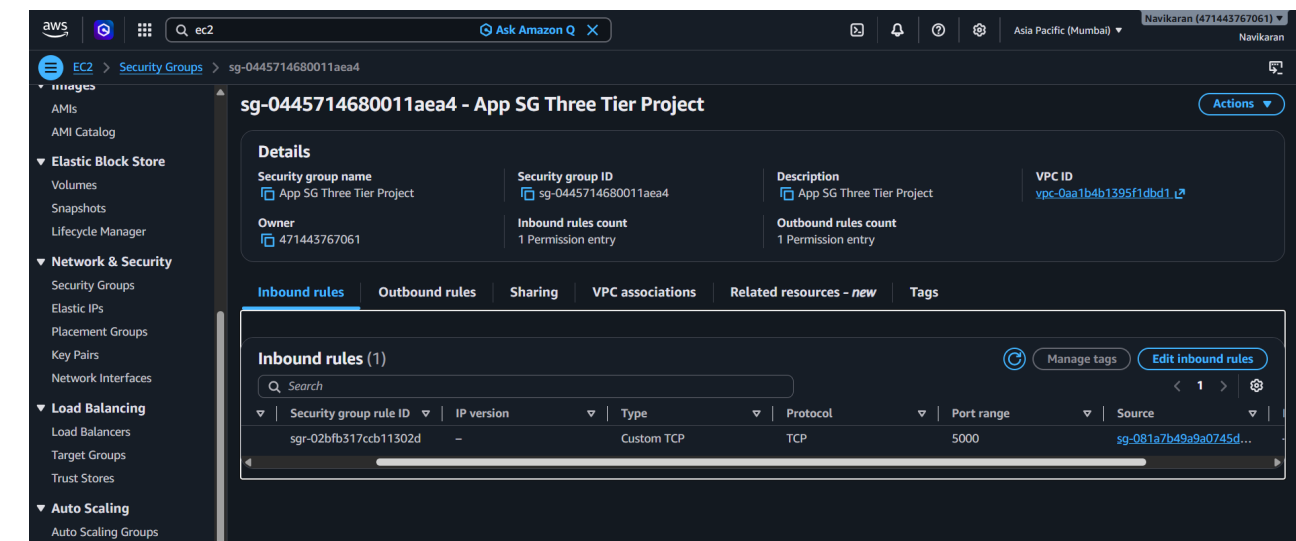
Search

< 1 >

Security group rule ID	IP version	Type	Protocol	Port range	Source
sgr-078a0465f5984f443	-	HTTP	TCP	80	sg-003879ce9dc2226cb...
sgr-0025a446f41a0eed1	IPv4	SSH	TCP	22	103.181.148.222/32

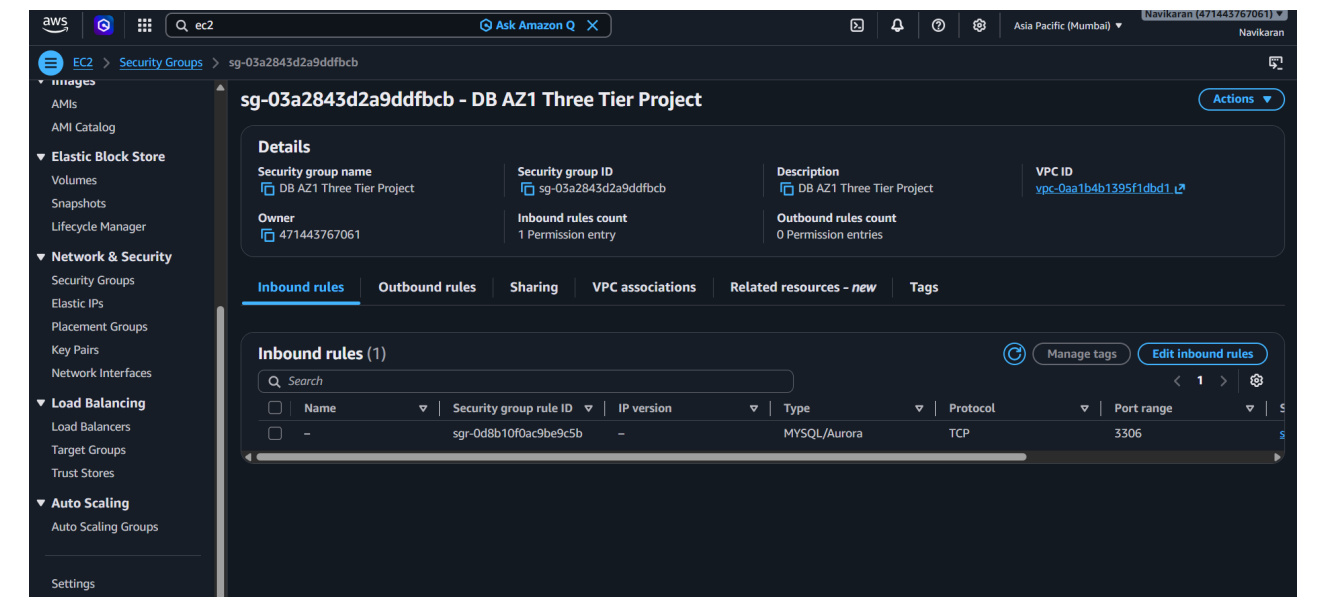
# Step 3 - App Tier Security Group

'App SG Three Tier Project' SG allows inbound Custom TCP 5000 only from the Internal-ALB-SG - App Tier only receives traffic through the internal load balancer.



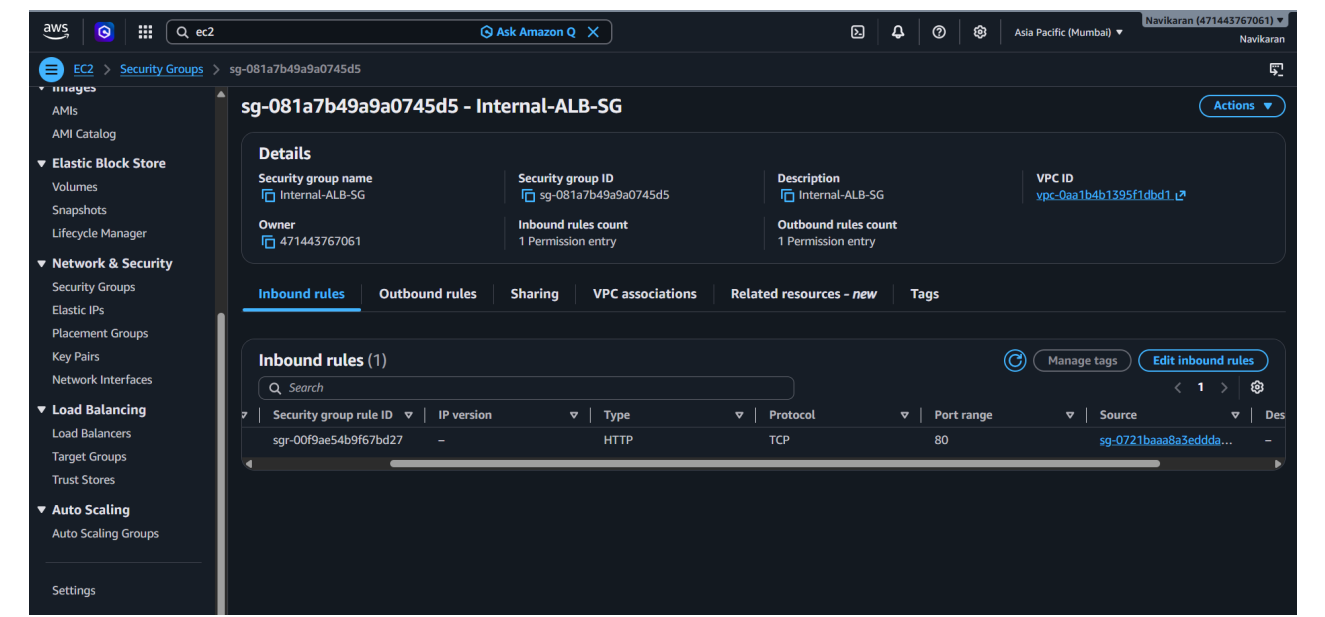
# Step 4 - Database Tier Security Group

'DB AZ1 Three Tier Project' SG allows inbound MySQL/Aurora (TCP 3306) - restricted so only the App Tier can reach the database.



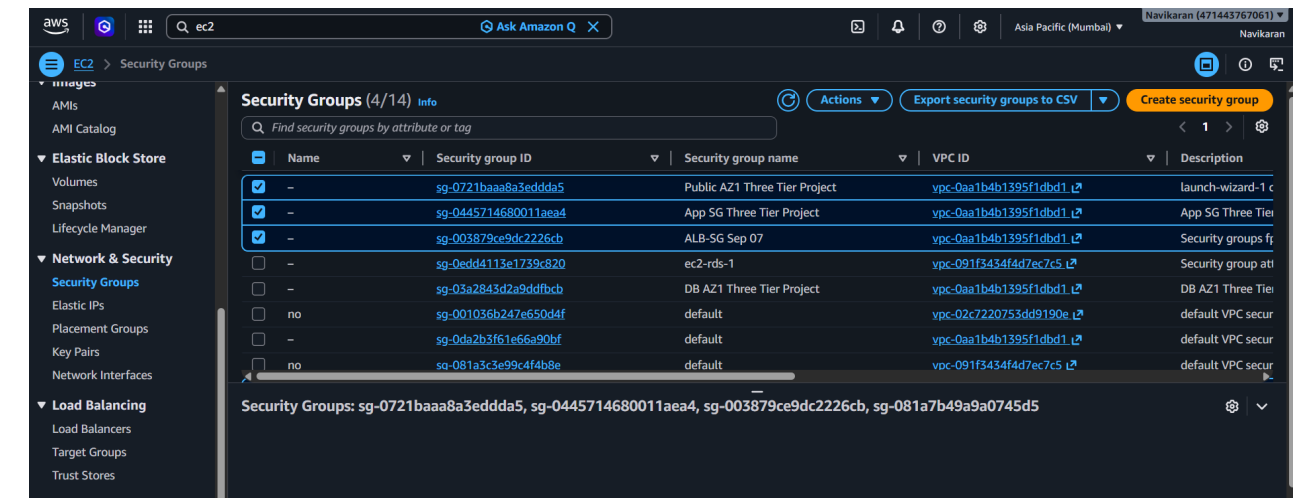
# Step 5 - Internal ALB Security Group

'Internal-ALB-SG' allows inbound HTTP (80) only from the Web Tier SG (Public AZ1) - keeping the internal load balancer reachable only from the Web Tier.



# Step 6 - All Security Groups Overview

Summary view of the security groups used across all tiers: Public AZ1 (Web), App SG, ALB-SG, and DB AZ1 - each scoped to allow traffic only from the tier in front of it.

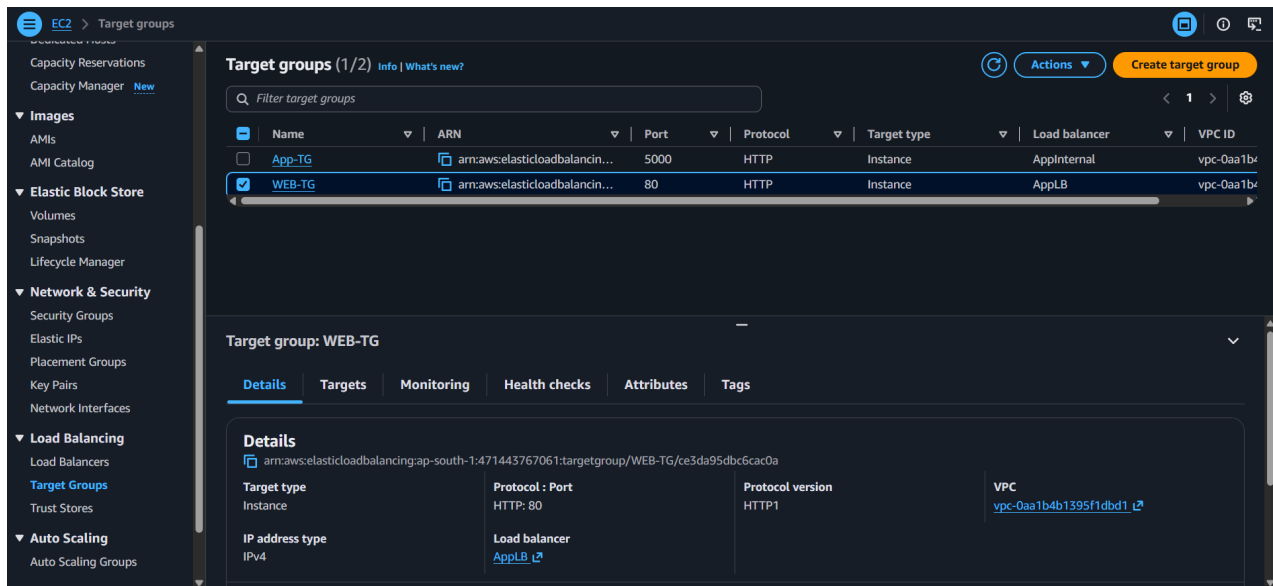


# AWS 3-Tier Architecture Project

## Task 6 & 7: Public and Internal Application Load Balancers

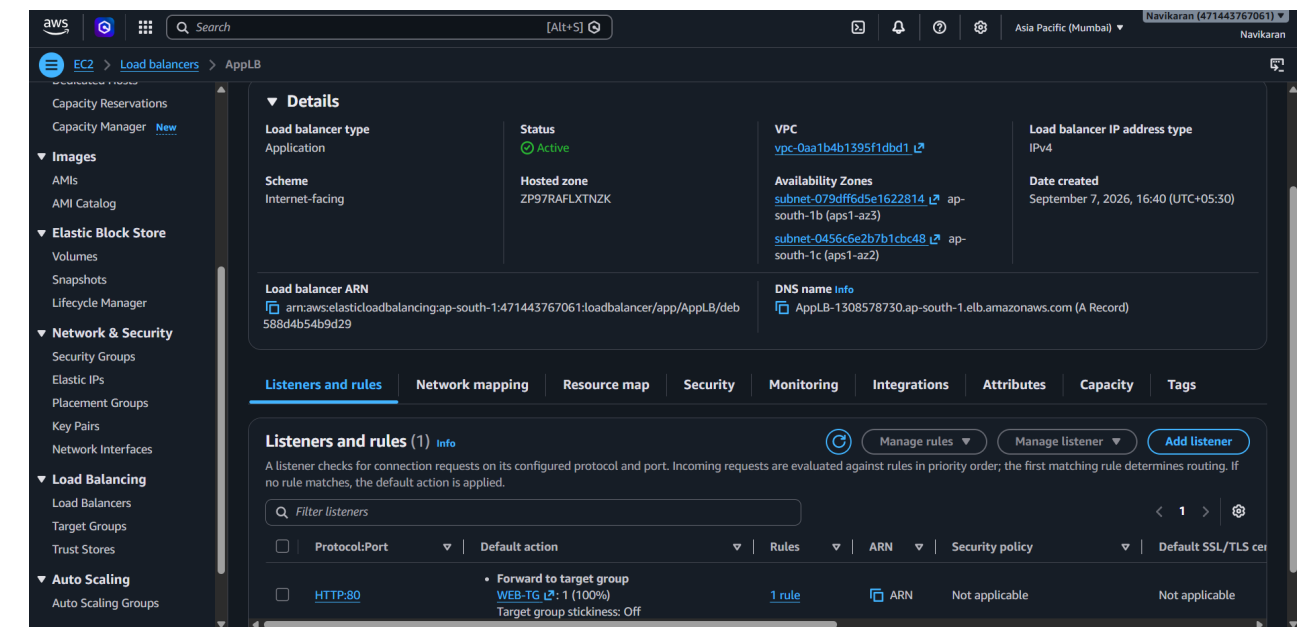
### Step 1 - Target Groups Created

Two target groups created: 'App-TG' (port 5000, attached to AppInternal) and 'WEB-TG' (port 80, attached to AppLB).



# Step 2 - Public ALB (AppLB) Configured

'AppLB' Application Load Balancer created - Internet-facing, Active status, listener HTTP:80 forwarding to WEB-TG.





## Step 3 - Web Target Group Healthy

WEB-TG shows 1/1 targets Healthy - the Web Tier EC2 instance (Public AZ1) is registered and passing health checks behind AppLB.

EC2 > Target groups
WEB-TG

- Capacity Reservations
- Capacity Manager New
- Images**
  - AMIs
  - AMI Catalog
- Elastic Block Store**
  - Volumes
  - Snapshots
  - Lifecycle Manager
- Network & Security**
  - Security Groups
  - Elastic IPs
  - Placement Groups
  - Key Pairs
  - Network Interfaces
- Load Balancing**
  - Load Balancers
  - Target Groups
  - Trust Stores
- Auto Scaling**
  - Auto Scaling Groups

am.amazonaws.com:elasticloadbalancing.ap-south-1:471443767061:targetgroup/WEB-TG/ce3da95dbcfac0a

<b>Target type</b> Instance	<b>Protocol : Port</b> HTTP: 80	<b>Protocol version</b> HTTP1
<b>IP address type</b> IPv4	<b>Load balancer</b> <a href="#">AppLB</a>	<b>VPC</b> <a href="#">vpc-0aa1b4b1395f1dbd1</a>

1  
Total targets

1  
Healthy  
0 Anomalous

0  
Unhealthy

0  
Unused

0  
Initial

0  
Draining

**Distribution of targets by Availability Zone (AZ)**

Select values in this table to see corresponding filters applied to the Registered targets table below.

**Targets**
Monitoring
Health checks
Attributes
Tags

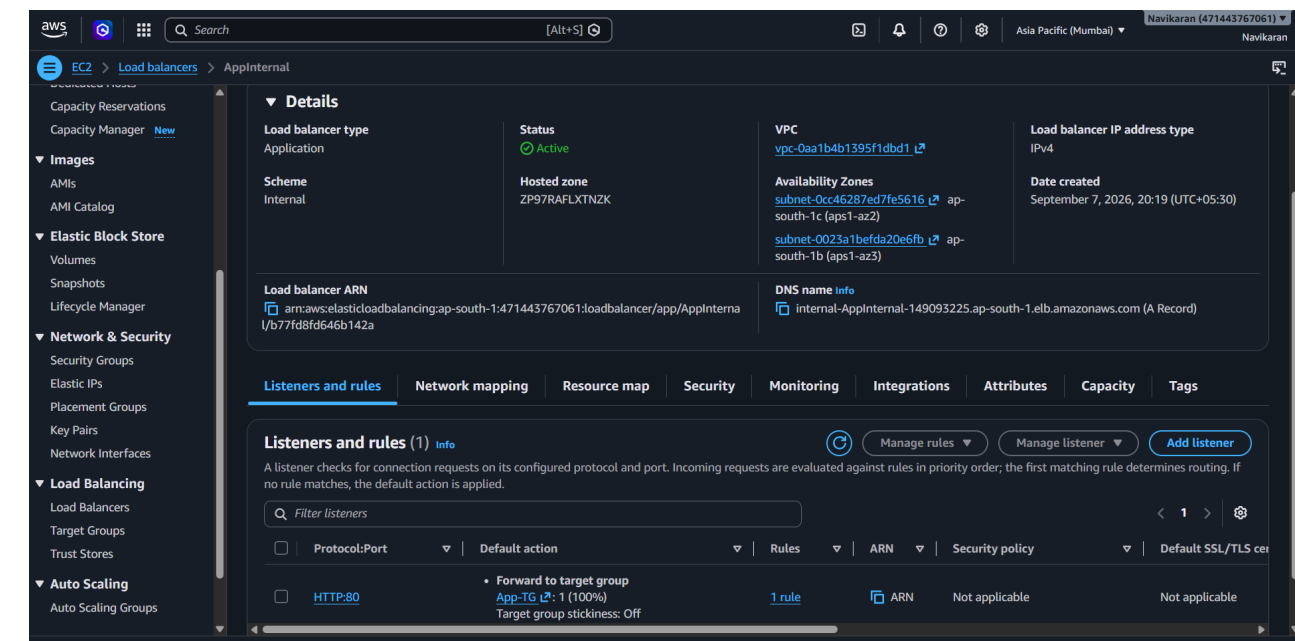
**Registered targets (1) Info** Anomaly mitigation: Not applicable Register Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

<input type="checkbox"/>	Instance ID	Name	Port	Zone	Health status	Health status details	Admini...	Overri...
<input type="checkbox"/>	<a href="#">i-07d2fab0e21fe5ba</a>	Public AZ1	80	ap-south-1c(a...)	Healthy	-	No override.	No over

# Step 4 - Internal ALB (AppInternal) Configured

'AppInternal' Application Load Balancer created - Internal scheme, Active status, listener HTTP:80 forwarding to App-TG.



# Step 5 - App Target Group Healthy

App-TG shows 1/1 targets Healthy - the App Tier EC2 instance (App AZ1, port 5000) is registered and passing health checks behind the Internal ALB.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Navikaran (471443767061)

Navikaran

EC2

Target groups

App-TG

Capacity Reservations

Capacity Manager

Images

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target groups

Trust Stores

Auto Scaling

Auto Scaling Groups

am:aws:elasticloadbalancing:ap-south-1:471443767061:targetgroup/App-TG/08e0ddd2809052f1

Target type

Instance

Protocol : Port

HTTP: 5000

Protocol version

HTTP1

VPC

vpc-0aa1b4b1395f1dbd1

IP address type

IPv4

Load balancer

AppInternal

1

Total targets

1

Healthy

0

Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (1)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

< 1 >

Instance ID

Name

Port

Zone

Health status

Health status details

Admini...

Overri...

i-070d82f7ce0d3c034

App AZ1

5000

ap-south-1c (a...

Healthy

-

No override.

No over

## Step 6 - Troubleshooting Health Checks

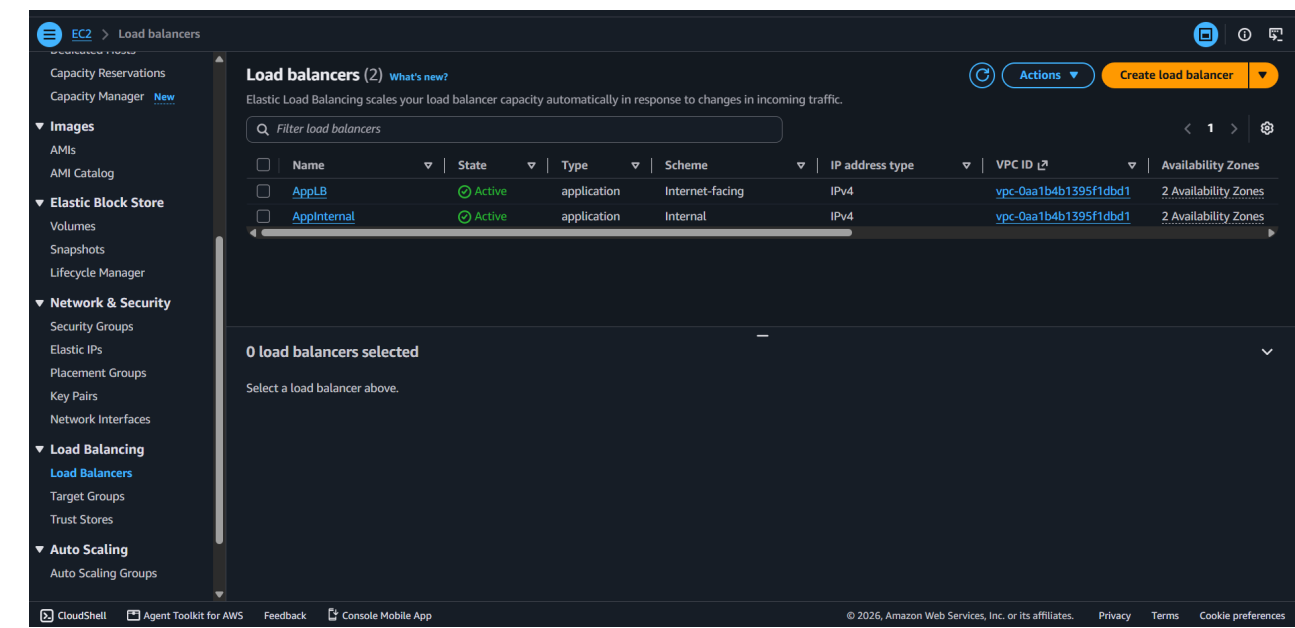
Initial health check requests to '/' returned 404. Added a dedicated '/health' route in the Flask app - health checks then returned 200 OK, confirmed via curl -i http://localhost:5000/health.

```
Press CTRL+C to quit
10.0.3.130 - - [07/Sep/2026 15:07:43] "GET / HTTP/1.1" 404 -
10.0.8.146 - - [07/Sep/2026 15:07:59] "GET / HTTP/1.1" 404 -
10.0.3.130 - - [07/Sep/2026 15:08:13] "GET / HTTP/1.1" 404 -
10.0.8.146 - - [07/Sep/2026 15:08:29] "GET / HTTP/1.1" 404 -
10.0.3.130 - - [07/Sep/2026 15:08:43] "GET / HTTP/1.1" 404 -
10.0.8.146 - - [07/Sep/2026 15:08:59] "GET / HTTP/1.1" 404 -
10.0.3.130 - - [07/Sep/2026 15:09:13] "GET / HTTP/1.1" 404 -
10.0.8.146 - - [07/Sep/2026 15:09:29] "GET / HTTP/1.1" 404 -
10.0.3.130 - - [07/Sep/2026 15:09:43] "GET / HTTP/1.1" 404 -
^C
sh-5.2$ 10.0.8.146 - - [07/Sep/2026 15:09:59] "GET / HTTP/1.1" 404 -
10.0.3.130 - - [07/Sep/2026 15:10:13] "GET / HTTP/1.1" 404 -
10.0.8.146 - - [07/Sep/2026 15:10:29] "GET / HTTP/1.1" 404 -
10.0.3.130 - - [07/Sep/2026 15:10:43] "GET / HTTP/1.1" 404 -
10.0.8.146 - - [07/Sep/2026 15:10:59] "GET / HTTP/1.1" 404 -
10.0.3.130 - - [07/Sep/2026 15:11:13] "GET /health HTTP/1.1" 200 -
10.0.8.146 - - [07/Sep/2026 15:11:29] "GET /health HTTP/1.1" 200 -
10.0.3.130 - - [07/Sep/2026 15:11:43] "GET /health HTTP/1.1" 200 -
10.0.8.146 - - [07/Sep/2026 15:11:59] "GET /health HTTP/1.1" 200 -
10.0.3.130 - - [07/Sep/2026 15:12:13] "GET /health HTTP/1.1" 200 -
^C
sh-5.2$ curl -i http://localhost:5000/health
127.0.0.1 - - [07/Sep/2026 15:12:27] "GET /health HTTP/1.1" 200 -
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.9.25
Date: Mon, 07 Sep 2026 15:12:27 GMT
Content-Type: application/json
Content-Length: 16
Connection: close

{"status":"ok"}
sh-5.2$ 10.0.8.146 - - [07/Sep/2026 15:12:29] "GET /health HTTP/1.1" 200 -
```

# Step 7 - Both Load Balancers Active

Load Balancers list confirms both 'AppLB' (Internet-facing) and 'AppInternal' (Internal) are Active, each spanning 2 Availability Zones.



# AWS 3-Tier Architecture Project

## Task 8 & 9: Nginx Reverse Proxy + Flask-RDS Integration Validation

### Step 1 - Configured Nginx Reverse Proxy

Edited /etc/nginx/conf.d/default.conf: root serves index.html at '/', and location /api/ set to proxy\_pass to the Internal ALB DNS (internal-AppInternal-149093225...elb.amazonaws.com).

```
GNU nano 8.3 /etc/nginx/conf.d/default.conf
server {
 listen 80;
 server_name _;

 root /usr/share/nginx/html;

 location / {
 index index.html;
 }

 location /api/ {
 proxy_pass http://internal-AppInternal-149093225.ap-south-1.elb.amazonaws.com/;
 }
}
```

## Step 2 - Validated & Restarted Nginx

nginx.conf syntax and configuration test successful, nginx.service (re)started. curl http://localhost on the Web Tier returns the page correctly.

```
Session ID: root-zb56c6o36q7jez7pt8kcx6nq24  Shortcuts Instance ID: f-07d2fab01e21fe5ba (Public AZ1)  Terminate session

CPU: 88ms
CGroup: /system.slice/nginx.service
├─47196 "nginx: master process /usr/sbin/nginx"
├─47197 "nginx: worker process"
└─47198 "nginx: worker process"

Sep 07 15:45:41 ip-10-0-1-125.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 07 15:45:41 ip-10-0-1-125.ap-south-1.compute.internal nginx[47193]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 07 15:45:41 ip-10-0-1-125.ap-south-1.compute.internal nginx[47193]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 07 15:45:41 ip-10-0-1-125.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
sh-5.2$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>
<p>For online documentation and support please refer to
nginx.org.

to engage with the community please visit
community.nginx.org.

For enterprise grade support, professional services, additional
security features and capabilities please refer to
f5.com/nginx.</p>
<p>Thank you for using nginx.</p>
</body>
</html>
sh-5.2$
```

## Step 3 - Feedback Form Calling API via Proxy

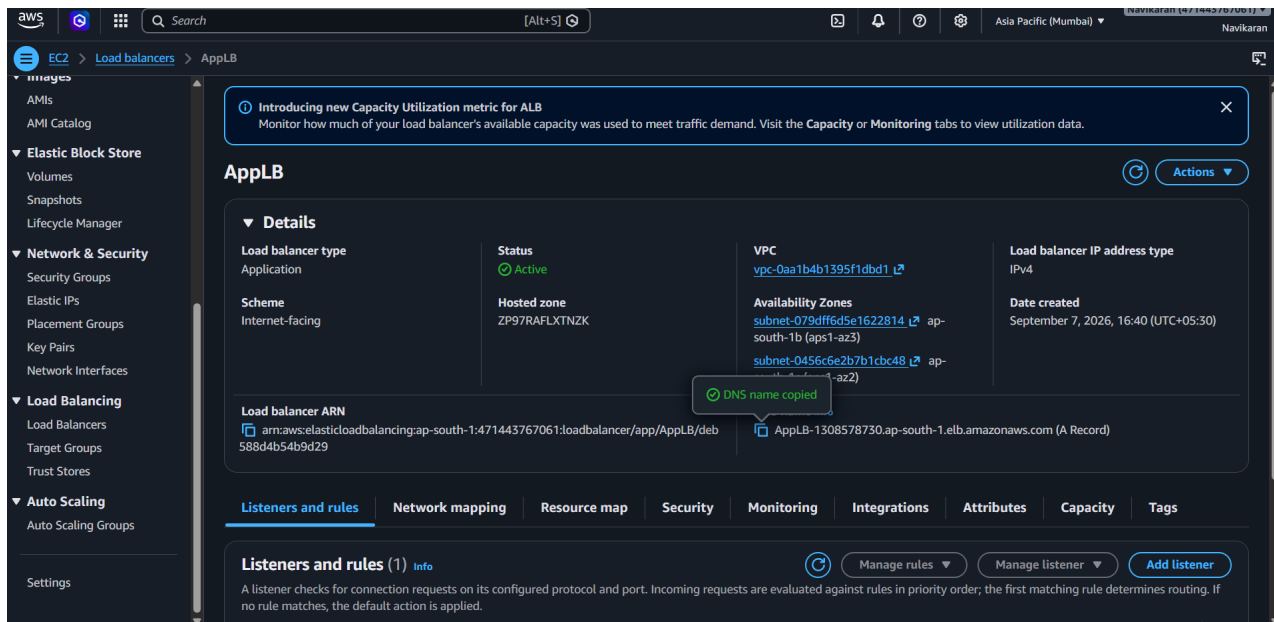
index.html Feedback Form uses fetch('/api/submit') with POST - Nginx forwards this request to the Internal ALB -> App Tier Flask app. Connection to AppLB DNS confirmed.

```
<
<!DOCTYPE html>
<html lang='en'>
<head>
 <meta charset='UTF-8'>
 <title>Feedback Form</title>
 <style>
 body { font-family: Arial, sans-serif; max-width: 500px; margin: 60px auto; }
 input, textarea { width:100%; padding:10px; margin:8px 0; box-sizing:border-box; }
 button { background:#2563EB; color:#fff; padding:12px 28px; border:none; cursor:pointer; }
 #msg { margin-top:12px; font-weight:bold; }
 </style>
</head>
<body>
 <h2>Feedback Form</h2>
 <input id='name' placeholder='Your Name' />
 <input id='email' placeholder='Your Email' type='email' />
 <textarea id='feedback' rows='4' placeholder='Your Feedback'></textarea>
 <button onclick='submit()'>Submit</button>
 <p id='msg'></p>
 <script>
 function submit() {
 const name = document.getElementById('name').value;
 const email = document.getElementById('email').value;
 const feedback = document.getElementById('feedback').value;
 fetch('/api/submit', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ name, email, feedback })
 })
 .then(r => r.json())
 .then(d => { document.getElementById('msg').innerText = d.message; })
 .catch(() => { document.getElementById('msg').innerText = 'Error submitting.'; });
 }
 </script>
</body>
</html>
* Connection #0 to host AppLB-1308578730.ap-south-1.elb.amazonaws.com:80 left intact
sh-5.2$
```



## Step 4 - Public ALB DNS for Browser Test

Copied the AppLB (Internet-facing) DNS name to test the full flow end-to-end in the browser: Public ALB -> Web Tier -> Nginx -> Internal ALB -> App Tier.



**Introducing new Capacity Utilization metric for ALB**  
Monitor how much of your load balancer's available capacity was used to meet traffic demand. Visit the **Capacity** or **Monitoring** tabs to view utilization data.

### AppLB

**Details**

<b>Load balancer type</b> Application	<b>Status</b> Active	<b>VPC</b> vpc-0aa1b4b1395f1dbd1	<b>Load balancer IP address type</b> IPv4
<b>Scheme</b> Internet-facing	<b>Hosted zone</b> ZP97RAFLXTNZK	<b>Availability Zones</b> subnet-079dff6d5e1622814 ap-south-1b (aps1-az3) subnet-0456c6e2b7b1cbc48 ap-south-1b (aps1-az2)	<b>Date created</b> September 7, 2026, 16:40 (UTC+05:30)

**Load balancer ARN**  
arn:aws:elasticloadbalancing:ap-south-1:471443767061:loadbalancer/app/AppLB/deb588d4b54b9d29

**AppLB-1308578730.ap-south-1.elb.amazonaws.com (A Record)**

**Listeners and rules (1)**

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

**Actions:** Manage rules, Manage listener, Add listener

## Step 5 - Verified Feedback Stored in RDS

Connected to the RDS endpoint and ran `SELECT * FROM feedback;` - multiple feedback entries submitted through the website (via Nginx proxy -> Flask -> RDS) are confirmed stored correctly.

```
sh-5.2$ mysql -h db.ctq00g6wq6xj.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MySQL connection id is 306
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> USE feedbackdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [feedbackdb]> SELECT * FROM feedback;
+----+-----+-----+-----+-----+
| id | name | email | feedback | created |
+----+-----+-----+-----+-----+
| 1 | Navi | navi@example.com | Testing from App EC2 | 2026-09-07 10:05:35 |
| 2 | Test | t@t.com | Hello RDS! | 2026-09-07 15:00:08 |
| 3 | Test | t@t.com | Hello RDS! | 2026-09-07 17:10:16 |
+----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

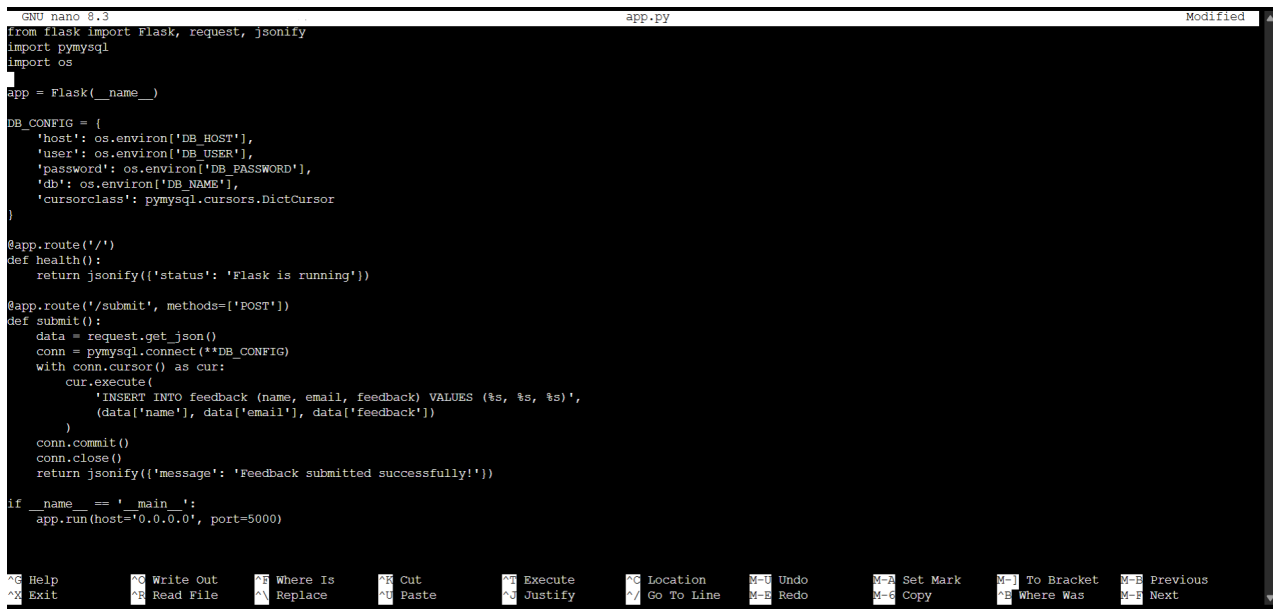
MySQL [feedbackdb]>
```

# AWS 3-Tier Architecture Project

## Task 10: Validate the Complete 3-Tier Architecture (End-to-End)

### Step 1 - app.py Using Environment Variables

Updated app.py to read DB connection details securely from environment variables (DB\_HOST, DB\_USER, DB\_PASSWORD, DB\_NAME) instead of hardcoding credentials.



```
GNU nano 8.3 app.py Modified
from flask import Flask, request, jsonify
import pymysql
import os

app = Flask(__name__)

DB_CONFIG = {
 'host': os.environ['DB_HOST'],
 'user': os.environ['DB_USER'],
 'password': os.environ['DB_PASSWORD'],
 'db': os.environ['DB_NAME'],
 'cursorclass': pymysql.cursors.DictCursor
}

@app.route('/')
def health():
 return jsonify({'status': 'Flask is running'})

@app.route('/submit', methods=['POST'])
def submit():
 data = request.get_json()
 conn = pymysql.connect(**DB_CONFIG)
 with conn.cursor() as cur:
 cur.execute(
 'INSERT INTO feedback (name, email, feedback) VALUES (%s, %s, %s)',
 (data['name'], data['email'], data['feedback'])
)
 conn.commit()
 conn.close()
 return jsonify({'message': 'Feedback submitted successfully!'})

if __name__ == '__main__':
 app.run(host='0.0.0.0', port=5000)
```

^G Help ^O Write Out ^F Where Is ^C Cut ^E Execute ^G Location ^U Undo ^M-A Set Mark ^M-] To Bracket ^M-B Previous  
^X Exit ^R Read File ^N Replace ^V Paste ^J Justify ^\_ Go To Line ^M-E Redo ^M-G Copy ^M-^ Where Was ^M-\_ Next

## Step 2 - Troubleshooting 404 Error

Hit a 404 'requested URL was not found' error. Checked routes with grep and cat on app.py to confirm the '/' and '/submit' routes were defined correctly.

```
<p>The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.</p>
sh-5.2$ grep -n "@app.route" app.py
15:@app.route('/')
19:@app.route('/submit', methods=['POST'])
sh-5.2$ cat app.py
from flask import Flask, request, jsonify
import pymysql
import os

app = Flask(__name__)

DB_CONFIG = {
 'host': os.environ['DB_HOST'],
 'user': os.environ['DB_USER'],
 'password': os.environ['DB_PASSWORD'],
 'db': os.environ['DB_NAME'],
 'cursorclass': pymysql.cursors.DictCursor
}

@app.route('/')
def health():
 return jsonify({'status': 'Flask is running'})

@app.route('/submit', methods=['POST'])
def submit():
 data = request.get_json()
 conn = pymysql.connect(**DB_CONFIG)
 with conn.cursor() as cur:
 cur.execute(
 'INSERT INTO feedback (name, email, feedback) VALUES (%s, %s, %s)',
 (data['name'], data['email'], data['feedback'])
)
 conn.commit()
 conn.close()
 return jsonify({'message': 'Feedback submitted successfully!'})

if __name__ == '__main__':
 app.run(host='0.0.0.0', port=5000)
sh-5.2$
```

## Step 3 - Fixing 'Port Already in Use'

curl to port 5000 still returned 404. Found the old process still bound to port 5000 using lsof, and worked on killing it to restart cleanly.

```
}
@app.route('/')
def health():
 return jsonify({'status': 'Flask is running'})

@app.route('/submit', methods=['POST'])
def submit():
 data = request.get_json()
 conn = pymysql.connect(**DB_CONFIG)
 with conn.cursor() as cur:
 cur.execute(
 'INSERT INTO feedback (name, email, feedback) VALUES (%s, %s, %s)',
 (data['name'], data['email'], data['feedback'])
)
 conn.commit()
 conn.close()
 return jsonify({'message': 'Feedback submitted successfully!'})

if __name__ == '__main__':
 app.run(host='0.0.0.0', port=5000)

sh-5.2$ curl http://localhost:5000
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.</p>
sh-5.2$ sudo lsof -i :5000
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
python3 43415 ssm-user 4u IPv4 82515 0t0 TCP *:complex-main (LISTEN)
sh-5.2$ kill -9 1234
sh: Kill: (1234) - No such process
sh-5.2$ python3 app.py &
[1] 50237
sh-5.2$ * Serving Flask app 'app'
* Debug mode: off
Address already in use
Port 5000 is in use by another program. Either identify and stop that program, or start the server with a different port.
```

## Step 4 - App Restarted Successfully

Killed the stale process (PID 43415), restarted with 'python3 app.py &' - Flask now serving correctly on http://10.0.3.225:5000, confirmed via curl {"status":"ok"}.

```
if __name__ == '__main__':
 app.run(host='0.0.0.0', port=5000)
sh-5.2$ curl http://localhost:5000
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.</p>
sh-5.2$ sudo lsof -i :5000
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
python3 43415 ssm-user 4u IPv4 82515 0t0 TCP *:complex-main (LISTEN)
sh-5.2$ kill -9 1234
sh: kill: (1234) - No such process
sh-5.2$ python3 app.py &
[1] 50237
sh-5.2$ * Serving Flask app 'app'
* Debug mode: off
Address already in use
Port 5000 is in use by another program. Either identify and stop that program, or start the server with a different port.
sh-5.2$ kill -9 43415
[1]+ Done(1) python3 app.py
sh-5.2$ kill -9 43415
sh: Kill: (43415) - No such process
sh-5.2$ sudo lsof -i :5000
sh-5.2$ python3 app.py &
[1] 50314
sh-5.2$ * Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.3.225:5000
Press CTRL+C to quit
10.0.3.130 - - [07/Sep/2026 18:21:51] "GET /health HTTP/1.1" 404 -
^C
sh-5.2$ curl http://localhost:5000
127.0.0.1 - - [07/Sep/2026 18:21:57] "GET / HTTP/1.1" 200 -
{"status":"Flask is running"}
sh-5.2$
```

## Step 5 - Setting DB Environment Variables

Hit a KeyError: 'DB\_HOST' since env vars weren't set in this new session. Exported DB\_HOST, DB\_USER, DB\_PASSWORD, DB\_NAME and verified with echo.

```
sh-5.2$ pwd
/home/ssm-user/Three-Tier-Architecture
sh-5.2$ ls -la
total 24
drwxr-xr-x. 4 ssm-user ssm-user 130 Sep 7 18:11 .
drwx-----. 6 ssm-user ssm-user 169 Sep 7 17:57 ..
drwxr-xr-x. 7 ssm-user ssm-user 147 Sep 7 07:06 .git
-rw-r--r--. 1 ssm-user ssm-user 1823 Sep 7 07:06 App.md
-rw-r--r--. 1 ssm-user ssm-user 2597 Sep 7 07:06 DB.md
drwxr-xr-x. 2 ssm-user ssm-user 32 Sep 7 18:11 __pycache__
-rw-r--r--. 1 ssm-user ssm-user 882 Sep 7 18:11 app.py
-rw-r--r--. 1 ssm-user ssm-user 1349 Sep 7 07:06 index.html
-rw-r--r--. 1 ssm-user ssm-user 619 Sep 7 07:06 nginx.conf
-rw-r--r--. 1 ssm-user ssm-user 2158 Sep 7 07:06 proxy.md
sh-5.2$ ls -l /home/ssm-user/Three-Tier-Architecture/app.py
-rw-r--r--. 1 ssm-user ssm-user 882 Sep 7 18:11 /home/ssm-user/Three-Tier-Architecture/app.py
sh-5.2$ Spython3 app.py
sh: Spython3: command not found
sh-5.2$ python3 app.py
Traceback (most recent call last):
 File "/home/ssm-user/Three-Tier-Architecture/app.py", line 8, in <module>
 'host': os.environ['DB_HOST'],
 File "/usr/lib64/python3.9/os.py", line 679, in __getitem__
 raise KeyError(key) from None
KeyError: 'DB_HOST'
sh-5.2$ cd /home/ssm-user/Three-Tier-Architecture
sh-5.2$ export DB_HOST="db.ctq00g6wq6xj.ap-south-1.rds.amazonaws.com"
sh-5.2$ export DB_USER="admin"
sh-5.2$ export DB_PASSWORD="8072075089"
sh-5.2$ export DB_NAME="feedbackdb"
sh-5.2$ echo "$DB_HOST"

sh-5.2$ echo "$DB_HOST"

sh-5.2$ echo "$DB_USER"
admin
sh-5.2$ echo "$DB_NAME"
feedbackdb
sh-5.2$
```

## Step 6 - App Running After Fixing Env Vars

Fixed a typo ('Spython3'), re-ran `python3 -m py_compile app.py` to check syntax, then started `app.py` successfully - running on `10.0.3.225:5000`.

```
sh-5.2$ Spython3 app.py
sh: Spython3: command not found
sh-5.2$ python3 app.py
Traceback (most recent call last):
 File "/home/ssm-user/Three-Tier-Architecture/app.py", line 8, in <module>
 'host': os.environ['DB_HOST']],
 File "/usr/lib64/python3.9/os.py", line 679, in __getitem__
 raise KeyError(key) from None
KeyError: 'DB_HOST'
sh-5.2$ cd /home/ssm-user/Three-Tier-Architecture
sh-5.2$ export DB_HOST="db.ctg00g6wq6xj.ap-south-1.rds.amazonaws.com"
sh-5.2$ export DB_USER="admin"
sh-5.2$ export DB_PASSWORD="8072075089"
sh-5.2$ export DB_NAME="feedbackdb"
sh-5.2$ echo "$DB_HOST"

sh-5.2$ echo "$DB_HOST"

sh-5.2$ echo "$DB_USER"
admin
sh-5.2$ echo "$DB_NAME"
feedbackdb
sh-5.2$ export DB_HOST="db.ctg00g6wq6xj.ap-south-1.rds.amazonaws.com"
sh-5.2$ echo "$DB_HOST"
db.ctg00g6wq6xj.ap-south-1.rds.amazonaws.com
sh-5.2$ echo "$DB_USER"
admin
sh-5.2$ echo "$DB_NAME"
feedbackdb
sh-5.2$ python3 -m py_compile app.py
sh-5.2$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.3.225:5000
Press CTRL+C to quit
```



## Step 7 - Connecting to RDS with Env Variables

Verified Flask app health via curl. Initial mysql connection attempt failed (missing -h host), then exported DB\_HOST/DB\_USER/DB\_PASSWORD/DB\_NAME and connected successfully to the MariaDB/MySQL RDS instance.

```
sh-5.2$ curl http://localhost:5000/
{"status": "Flask is running"}
sh-5.2$ mysql -h "$DB_HOST" -u "$DB_USER" -p
Enter password:
ERROR 2002 (HY000): Can't connect to local MySQL server through socket '/var/lib/mysql/mysql.sock' (2)
sh-5.2$ mysql -h "$DB_HOST" -u "$DB_USER" -p
Enter password:
ERROR 2002 (HY000): Can't connect to local MySQL server through socket '/var/lib/mysql/mysql.sock' (2)
sh-5.2$ echo "$DB_HOST"

sh-5.2$ export DB_HOST="db.ctq00g6wg6xj.ap-south-1.rds.amazonaws.com"
sh-5.2$ export DB_USER="admin"
sh-5.2$ export DB_PASSWORD="8072075089"
sh-5.2$ export DB_NAME="feedbackdb"
sh-5.2$ echo "$DB_HOST"
db.ctq00g6wg6xj.ap-south-1.rds.amazonaws.com
sh-5.2$ mysql -h "$DB_HOST" -u "$DB_USER" -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MySQL connection id is 637
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

## Step 8 - Listing Databases

Fixed a syntax typo (SHOW DATABASE -> SHOW DATABASES) and confirmed 'feedbackdb' exists among the RDS databases.

```
Enter password:
ERROR 2002 (HY000): Can't connect to local MySQL server through socket '/var/lib/mysql/mysql.sock' (2)
sh-5.2$ echo "$DB_HOST"

sh-5.2$ export DB_HOST="db.ctg00g6wg6xj.ap-south-1.rds.amazonaws.com"
sh-5.2$ export DB_USER="admin"
sh-5.2$ export DB_PASSWORD="8072075089"
sh-5.2$ export DB_NAME="feedbackdb"
sh-5.2$ echo "$DB_HOST"
db.ctg00g6wg6xj.ap-south-1.rds.amazonaws.com
sh-5.2$ mysql -h "$DB_HOST" -u "$DB_USER" -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MySQL connection id is 637
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> SHOW DATABASE;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'DATABASE' at line 1
MySQL [(none)]> SHOW DATABASES;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'DATABASE' at line 1
MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| feedbackdb |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.001 sec)

MySQL [(none)]>
```

## Step 9 - Selected feedbackdb & Listed Tables

Ran USE feedbackdb; then SHOW TABLES; - confirmed the 'feedback' table exists in the database.

```
Your MySQL connection id is 637
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> SHOW DATABASE;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'DATABASE' at line 1
MySQL [(none)]> SHOW DATABASE;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'DATABASE' at line 1
MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| feedbackdb |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.001 sec)

MySQL [(none)]> USE feedbackdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [feedbackdb]> SHOW TABLES;
+-----+
| Tables_in_feedbackdb |
+-----+
| feedback |
+-----+
1 row in set (0.001 sec)

MySQL [feedbackdb]>
```

## Step 10 - Verified Table Data

Ran DESCRIBE feedback; to confirm schema, then SELECT \* FROM feedback; showing 3 existing feedback rows stored correctly from earlier tests.

```
5 rows in set (0.001 sec)

MySQL [(none)]> USE feedbackdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [feedbackdb]> SHOW TABLES;
+-----+
| Tables_in_feedbackdb |
+-----+
| feedback |
+-----+

1 row in set (0.001 sec)

MySQL [feedbackdb]> DESCRIBE feedback;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int | NO | PRI | NULL | auto_increment |
| name | varchar(100) | YES | | NULL | |
| email | varchar(100) | YES | | NULL | |
| feedback | text | YES | | NULL | |
| created | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+

5 rows in set (0.001 sec)

MySQL [feedbackdb]> SELECT * FROM feedback;
+-----+-----+-----+-----+-----+
| id | name | email | feedback | created |
+-----+-----+-----+-----+-----+
| 1 | Navi | navi@example.com | Testing from App EC2 | 2026-09-07 10:05:35 |
| 2 | Test | t@t.com | Hello RDS! | 2026-09-07 15:00:08 |
| 3 | Test | t@t.com | Hello RDS! | 2026-09-07 17:10:16 |
+-----+-----+-----+-----+-----+

3 rows in set (0.001 sec)

MySQL [feedbackdb]>
```

# Step 11 - Full End-to-End Browser Test

Opened the Public ALB DNS (applb-1308578730...elb.amazonaws.com) in the browser, filled the Feedback Form describing the full 3-tier data flow, clicked Submit - 'Feedback submitted successfully!' confirmed.



## Feedback Form

Name

navi@test.com

Testing full 3-tier AWS architecture  
Browser  
↓  
Public ALB

Submit

Feedback submitted successfully!

# Step 12 - Final Validation: Data Confirmed in RDS

Queried SELECT \* FROM feedback; again - the new row (Navi, navi@test.com, describing Browser -> Public ALB -> Web EC2 + Nginx -> Internal ALB -> App EC2 + Flask -> RDS MySQL) is confirmed stored, proving the complete 3-tier architecture works end-to-end.

```
MySQL [(none)]> USE feedbackdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [feedbackdb]> SELECT * FROM feedback;
+-----+-----+-----+-----+
| id | name | email | feedback |
+-----+-----+-----+-----+
| 1 | Navi | navi@example.com | Testing from App EC2 |
| 2 | Test | tst.com | Hello RDS! |
| 3 | Test | tst.com | Hello RDS! |
| 4 | Final Test | final@test.com | Testing full application |
| 5 | Navi | navi@test.com | Testing full 3-tier AWS architecture |
+-----+-----+-----+-----+

Browser
↓
Public ALB
↓
Web EC2 + Nginx
↓
Internal ALB
↓
App EC2 + Flask
↓
RDS MySQL | 2026-09-08 05:11:22 |
+-----+-----+-----+-----+
5 rows in set (0.001 sec)

MySQL [feedbackdb]>
```

# AWS 3-Tier Architecture Project

GitHub Submission & Final Live Testing

## Step 1 - Organizing Screenshots for Submission

Copied all project screenshots into the local 'navi' repo's screenshots/ folder and confirmed 73 files ready to commit and push to GitHub.

```
'Load Balancer Public ap - Copy - Copy.png'
'Load Balancer Public ap - Copy.png'
'Load Balancer Public ap.png'
'Load Balancers - Copy - Copy.png'
'Load Balancers - Copy.png'
'Load Balancers.png'
'MYSQL - Copy - Copy.png'
'MYSQL - Copy.png'
'MYSQL app.py - Copy - Copy.png'
'MYSQL app.py - Copy.png'
'MYSQL app.py.png'
MYSQL.png
NAT.png
'Nginx running health for web targets - Copy - Copy.png'
'Nginx running health for web targets - Copy.png'
'Nginx running health for web targets.png'
'RDS MYSQL.png'
'RDS Subnet groups.png'
'RDS endpoint.png'
'Route Table 1.png'
'Route Table 2.png'
'Route Table 3.png'
'Screenshot 2026-08-27 123615 - Copy - Copy.png'
'Screenshot 2026-08-27 123615 - Copy.png'
'Screenshot 2026-08-27 123615.png'
'Screenshot 2026-08-31 114348 - Copy (2).png'
navikaran@Navikaran-KTG:~/navi$ cp /mnt/c/Users/Navikaran/Downloads/TransferNow-20260901WZuFHAyB/*.png screenshots/
navikaran@Navikaran-KTG:~/navi$ ls screenshots | wc -l
73
navikaran@Navikaran-KTG:~/navi$ |
```

```
'Screenshot 2026-09-01 151633 - Copy (2).png'
'Screenshot 2026-09-01 151633 - Copy - Copy.png'
'Screenshot 2026-09-01 151633 - Copy.png'
'Screenshot 2026-09-01 151633.png'
'Screenshot 2026-09-01 151711 - Copy.png'
'Screenshot 2026-09-01 151711.png'
'Screenshot 2026-09-01 153121 - Copy.png'
'Screenshot 2026-09-01 153121.png'
'Screenshot 2026-09-01 153208.png'
'Screenshot 2026-09-01 153344 - Copy - Copy.png'
'Screenshot 2026-09-01 153344 - Copy.png'
'Screenshot 2026-09-01 153344.png'
'Security Groups.png'
Subnets.png
'Target Group app health.png'
'Target Groups.png'
'Testing Flask.png'
VPC.png
'Web targets health.png'
apt-update-vs-apt-upgrade-1-1.png
'install nginx running - Copy - Copy.png'
'install nginx running - Copy.png'
'install nginx running.png'
task1
task2
```

## Step 2 - Git Commit & Push Troubleshooting

Hit 'Author identity unknown' and then 'Authentication failed' errors while pushing. Fixed by setting git config user identity and using the correct GitHub username/token for authentication.

```
hint:
hint: git rm --cached ref-repo
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
navikaran@Navikaran-KTG:~/navi$ git commit -m "Final submission: 3-tier AWS architecture project"
Author identity unknown

*** Please tell me who you are.

Run

 git config --global user.email "you@example.com"
 git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: empty ident name (for <navikaran@Navikaran-KTG.localdomain>) not allowed
navikaran@Navikaran-KTG:~/navi$ git push origin main
Username for 'https://github.com': naviktg
Password for 'https://naviktg@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/naviktg/navi.git/'
navikaran@Navikaran-KTG:~/navi$
navikaran@Navikaran-KTG:~/navi$ git push origin main
Username for 'https://github.com': navi
Password for 'https://navi@github.com':
Everything up-to-date
navikaran@Navikaran-KTG:~/navi$ |
```



## Step 3 - Final Push to GitHub Successful

git push origin main completed successfully - 38 objects (4.24 MiB) pushed to <https://github.com/naviktg/navi.git>, main branch updated (8d49be4..e0e4ad0).

```
create mode 100755 screenshots/install nginx running.png
navikaran@Navikaran-KTG:~/navi$ git push origin main
Username for 'https://github.com': naviktg
Password for 'https://naviktg@github.com':
Enumerating objects: 39, done.
Counting objects: 100% (39/39), done.
Delta compression using up to 24 threads
Compressing objects: 100% (38/38), done.
Writing objects: 100% (38/38), 4.24 MiB | 1.73 MiB/s, done.
Total 38 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/naviktg/navi.git
 8d49be4..e0e4ad0 main -> main
navikaran@Navikaran-KTG:~/navi$ |
```

## Step 4 - Final Flask/RDS Troubleshooting Before Demo

One more round of verifying environment variables (DB\_HOST, DB\_USER, DB\_PASSWORD, DB\_NAME) and confirming the Flask app runs cleanly on 10.0.3.225:5000 with health checks passing (200 OK).

```
Traceback (most recent call last):
 File "/home/ssm-user/Three-Tier-Architecture/app.py", line 8, in <module>
 'host': os.environ['DB_HOST'],
 File "/usr/lib64/python3.9/os.py", line 679, in __getitem__
 raise KeyError(key) from None
KeyError: 'DB_HOST'
sh-5.2$ cd /home/ssm-user/Three-Tier-Architecture
sh-5.2$ export DB_HOST="db.ctq00g6wq6xj.ap-south-1.rds.amazonaws.com"
sh-5.2$ export DB_USER="admin"
sh-5.2$ export DB_PASSWORD="8072075089"
sh-5.2$ export DB_NAME="feedbackdb"
sh-5.2$ echo "$DB_HOST"

sh-5.2$ echo "$DB_HOST"

sh-5.2$ echo "$DB_USER"
admin
sh-5.2$ echo "$DB_NAME"
feedbackdb
sh-5.2$ export DB_HOST="db.ctq00g6wq6xj.ap-south-1.rds.amazonaws.com"
sh-5.2$ echo "$DB_HOST"
db.ctq00g6wq6xj.ap-south-1.rds.amazonaws.com
sh-5.2$ echo "$DB_USER"
admin
sh-5.2$ echo "$DB_NAME"
feedbackdb
sh-5.2$ python3 -m py_compile app.py
sh-5.2$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.3.225:5000
Press CTRL+C to quit
10.0.8.146 - - [08/Sep/2026 04:57:36] "GET /health HTTP/1.1" 404 -
10.0.3.130 - - [08/Sep/2026 04:57:50] "GET /health HTTP/1.1" 404 -
10.0.8.146 - - [08/Sep/2026 04:58:06] "GET /health HTTP/1.1" 404 -
127.0.0.1 - - [08/Sep/2026 04:58:07] "GET / HTTP/1.1" 200 -
```

# Step 5 - Final Live Browser Test

Opened the Public ALB DNS (applb-1308578730...elb.amazonaws.com) in the browser, submitted a fresh feedback entry ('Navi' / 'web working') - 'Feedback submitted successfully!' confirms the entire 3-tier application is live and working end-to-end.

